



دانشکده مهندسی کامپیوتر

بررسی مدل‌های زبانی بزرگ عامل محور در زمینه جستجوی معماری شبکه‌های عصبی و بهینه سازی ابریارامتر

گزارش سمینار کارشناسی ارشد

در رشته مهندسی کامپیوتر گرایش هوش مصنوعی و رباتیک

محمدصادق پولائی موزیرجی

اساتید راهنما

دکتر محمدرضا محمدی و دکتر سید صالح اعتمادی

آبان ماه ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده

رویکردهای سنتی یادگیری ماشین خودکار با استفاده از بهینه‌سازی بیزی و الگوریتم‌های تکاملی فاقد تفسیرپذیری بوده و در بهره‌برداری از دانش حوزه‌ای دچار چالش هستند. مدل‌های زبانی بزرگ با توانایی‌های درک زبان طبیعی، استدلال و تولید کد، تحولی بنیادین ارائه می‌دهند. این سمینار بررسی می‌کند چگونه عامل‌های مبتنی بر مدل‌های زبانی بزرگ می‌توانند سیستم‌های یادگیری ماشین خودکار را با عمل به عنوان عامل‌های هوشمند، آگاه از زمینه و تطبیق‌پذیر دگرگون سازند. این پژوهش یک مرور نظام‌مند از رویکردهای اخیر از سه منظر ارائه می‌دهد: (۱) معماری عامل: سامانه‌های تک‌عاملی (بهینه‌سازی مستقیم، عملگرهای تکاملی، کنترل‌گرهای جریان) در مقابل چندعاملی (همکاری مبتنی بر نقش، هماهنگی سلسله‌مراتبی) (۲) منابع دانش: دانش درونی (تاریخچه بهینه‌سازی، یادگیری درون‌متنی) در برابر بازیابی بیرونی (استخراج از ادبیات، مخازن مدل) و (۳) قالب خروجی: پیکربندی‌های ساختاریافته، تولید کد، نمایش‌های درختی و رویکردهای ترکیبی. تحلیل نشان می‌دهد حوزه به طور مساوی بین معماری‌های تک‌عاملی و چندعاملی تقسیم شده، با تکیه اکثر سامانه‌ها بر دانش درونی و بهره‌گیری اندک از دانش بیرونی. قالب‌های خروجی تنوع بالایی دارند و رویکردهای ترکیبی تعادل بین خوانایی ماشینی و بیان‌پذیری را فراهم می‌کنند. سمینار مسائل باز و جهت‌های آینده شامل بهینه‌سازی با منابع محدود، یکپارچه‌سازی دانش پیشرفته و چارچوب‌های چندعاملی را شناسایی می‌کند. پیشنهاد پایان‌نامه برای طراحی سیستم عامل-محور جهت انتخاب و تنظیم مدل در انتقال یادگیری با داده محدود ارائه می‌شود.

واژگان کلیدی: یادگیری ماشین خودکار، مدل‌های زبانی بزرگ، سیستم‌های عامل-محور، بهینه‌سازی ابرپارامترها، جستجوی معماری عصبی، تولید تقویت‌شده با بازیابی، یادگیری درون‌متنی، انتقال یادگیری

فهرست مطالب

چ	فهرست شکل‌ها
ح	فهرست جداول
۱	فصل ۱: مقدمه
۱	۱-۱ شرح مسأله
۲	۲-۱ معرفی حوزه سمینار
۳	۳-۱ ساختار گزارش
۴	فصل ۲: تعاریف و مفاهیم مبنایی
۴	۱-۲ مقدمه
۴	۲-۲ یادگیری خودکار ماشین
۶	۱-۲-۲ بهینه‌سازی ابرپارامتر
۸	۲-۲-۲ جستجوی معماری شبکه عصبی
۸	۳-۲ مدل‌های زبانی بزرگ
۸	۱-۳-۲ تاریخچه
۹	۲-۳-۲ معماری
۹	۳-۳-۲ کاربردها
۱۱	۴-۳-۲ تفکر و عمل
۱۱	۴-۲ عامل
۱۱	۱-۴-۲ تک‌عاملی

۱۲	۲-۴-۲ چندعاملی
۱۳	۵-۲ تولید تقویت شده با بازیابی
۱۴	۲-۵-۱ پایگاه دانش
۱۵	۲-۵-۲ ترکیب با عامل
۱۶	فصل ۳: مروری بر کارهای مرتبط
۱۶	۱-۳ مقدمه
۱۶	۲-۳ ظهور نخستین روش های یادگیری ماشین خودکار
۱۹	۳-۳ تحلیل بر مبنای معماری عامل
۱۹	۱-۳-۳ سامانه های تک عاملی
۲۲	۲-۳-۳ سامانه های چندعاملی
۲۴	۴-۳ تحلیل منابع دانش
۲۴	۱-۴-۳ دانش درونی: تاریخچه آزمون و بازتاب
۲۷	۲-۴-۳ دانش بیرونی: بازیابی از ادبیات و مخازن
۲۹	۳-۴-۳ راهبردهای تقویت آمیخته
۳۰	۵-۳ تحلیل بر مبنای قالب خروجی مدل
۳۰	۱-۵-۳ خروجی های سبک واژه نامه ای
۳۰	۲-۵-۳ تولید کد برنامه
۳۲	۳-۵-۳ خروجی های درخت ساختار
۳۳	۴-۵-۳ خروجی های ترکیبی
۳۵	۶-۳ طبقه بندی کار های مرتبط
۳۶	فصل ۴: نتیجه گیری و کارهای آینده
۳۶	۱-۴ نتیجه گیری
۳۷	۲-۴ مسائل باز و کارهای قابل انجام
۳۹	۳-۴ معرفی موضوع مورد نظر برای پایان نامه

فهرست مطالب

ج

کتاب‌نامه

۴۲

واژه‌نامه فارسی به انگلیسی

۴۷

واژه‌نامه انگلیسی به فارسی

۵۳

فهرست شکل‌ها

۵	۱-۲ چارچوب یادگیری ماشین خودکار
۱۰	۲-۲ معماری ترنسفورمر
۱۲	۳-۲ چارچوب ReAct
۱۴	۴-۲ چارچوب کلی تولید تقویت‌شده با بازایی
۱۷	۱-۳ دسته بندی مقالات
۲۰	۲-۳ مدل زبانیها برای بهینه‌سازی ابرپارامترها
۲۱	۳-۳ روند روش تکاملی
۲۳	۴-۳ چارچوب مبتنی بر نقش
۲۵	۵-۳ یادگیری درون‌متنی از تاریخچه بهینه‌سازی
۲۹	۶-۳ چارچوب HuggingGPT برای خودکارسازی یادگیری ماشین
۳۱	۷-۳ نمونه ای از خروجی قالب واژه‌نامه‌ای
۳۱	۸-۳ نمونه ای از خروجی تولید کد
۳۳	۹-۳ نمونه ای از خروجی درخت ساختار
۳۴	۱۰-۳ نمونه ای از خروجی ترکیبی واژه‌نامه‌ای و کد

فهرست جداول

۵	۱-۲ مراحل و وظایف کلیدی در چارچوب یادگیری ماشین خودکار
۷	۲-۲ مقایسه روش‌های بهینه‌سازی ابرپارامتر
۳۵	۱-۳ مقایسه فشرده مقالات مبتنی بر LLM

فصل ۱

مقدمه

۱-۱ شرح مسأله

فرایند ساخت و بهینه‌سازی مدل‌های یادگیری ماشین به‌طور سنتی، فرایندی پیچیده، زمان‌بر و نیازمند دانش تخصصی عمیق است. متخصصان علم داده ناچارند زمان قابل توجهی را صرف مهندسی ویژگی^۱، انتخاب مدل^۲، و تنظیم ابرپارامترها^۳ کنند. یادگیری ماشین خودکار^۴ با هدف خودکارسازی این زنجیره وظایف^۵ و مردمی‌سازی^۶ یادگیری ماشین پدیدار شد. با این حال، روش‌های سنتی یادگیری ماشین خودکار، مانند بهینه‌سازی بیزی^۷ یا الگوریتم‌های تکاملی^۸، اغلب به عنوان بهینه‌سازهای جعبه‌سیاه^۹ عمل می‌کنند که فاقد تفسیرپذیری بوده و در انطباق با مسائل جدید یا بهره‌برداری از دانش برون‌حوزه‌ای دچار چالش هستند. در سال‌های اخیر، ظهور مدل‌های زبانی بزرگ^{۱۰} با توانایی‌های چشمگیر در درک زبان

^۱feature engineering

^۲model selection

^۳hyperparameter tuning

^۴Automated Machine Learning (AutoML)

^۵pipeline

^۶democratization

^۷Bayesian optimization

^۸evolutionary algorithms

^۹black-box

^{۱۰}Large Language Models (LLMs)

طبیعی^{۱۱}، استدلال و تولید کد، روش جدیدی را معرفی کرده است. این مدل‌ها پتانسیل آن را دارند که از بهینه‌سازهای کور فراتر رفته و به عنوان عامل^{۱۲} های هوشمند عمل کنند. این عامل‌ها می‌توانند با استفاده از دانش پیشین خود، استدلال گام‌به‌گام، و حتی تعامل با محیط‌های اجرایی و پایگاه‌های دانش^{۱۳}، فرایند یادگیری ماشین خودکار را به شیوه‌ای خودمختار^{۱۴}، تطبیق‌پذیر^{۱۵} و آگاه از بافتار^{۱۶} هدایت کنند. مسئله اصلی که این سمینار به آن می‌پردازد، بررسی این تقاطع نوظهور است: چگونه می‌توان از مدل‌های زبانی بزرگ عامل-محور برای دگرگونی و ارتقای نسل بعدی سیستم‌های یادگیری ماشین خودکار بهره جست؟

۲-۱ معرفی حوزه سمینار

حوزه اصلی این سمینار، یادگیری ماشین خودکار است. این حوزه به طور سنتی بر توسعه روش‌هایی برای خودکارسازی کامل خط لوله یادگیری ماشین، به‌ویژه چالش‌های محاسباتی پیچیده‌ای چون بهینه‌سازی ابرپارامترها^{۱۷} و جستجوی معماری عصبی^{۱۸}، تمرکز داشته است. این سمینار یک روش نوظهور و دگرگون‌ساز در این حوزه را بررسی می‌کند که مبتنی بر ادغام مدل‌های زبانی بزرگ است. برخلاف روش‌های کلاسیک یادگیری ماشین خودکار (مانند بهینه‌سازی بیزی یا الگوریتم‌های تکاملی) که اغلب به عنوان بهینه‌سازهای جعبه‌سیاه عمل می‌کنند، مدل‌های زبانی بزرگ با توانایی‌های بی‌نظیر خود در درک دستورالعمل‌های پیچیده، استدلال گام‌به‌گام و تولید کد، پتانسیل ایجاد فرایندهای بهینه‌سازی شفاف‌تر، انعطاف‌پذیرتر و آگاهانه‌تر را فراهم می‌کنند.

نکته کلیدی که این سمینار به آن می‌پردازد، بررسی این مدل‌های زبانی در چارچوب

¹¹ Natural Language Understanding (NLU)

¹² agent

¹³ knowledge bases

¹⁴ autonomous

¹⁵ adaptive

¹⁶ context-aware

¹⁷ Hyperparameter Optimization (HPO)

¹⁸ Neural Architecture Search (NAS)

سیستم‌های عامل – محور^{۱۹} است. در این دیدگاه، مدل زبانی به عنوان مغز متفکر یک عامل هوشمند عمل می‌کند. این عامل می‌تواند به صورت خودمختار، وظایف پیچیده یادگیری ماشین خودکار را مدیریت کند، از دانش خارجی بهره‌برد، استراتژی‌های جستجو را استدلال نماید، کد اجرایی تولید کند و بر اساس بازخورد، رویکرد خود را تطبیق دهد. بنابراین، حوزه تخصصی این سمینار، نقطه تلاقی این مفاهیم مطالعه و تحلیل کاربرد عامل‌های هوشمند مبتنی بر مدل زبانی بزرگ به منظور ارتقا و تحول در نسل جدید سیستم‌های یادگیری ماشین خودکار می‌باشد.

۱-۳ ساختار گزارش

این گزارش در ۴ فصل تهیه شده است. در فصل اول به بیان مقدمه و شرح مسئله پرداخته ایم. در فصل دوم مبانی و مفاهیم حوزه یادگیری ماشین خودکار، بهینه‌سازی ابرپارامترها و مدل‌های زبانی بزرگ عامل محور شرح داده شده است. در فصل سوم روش‌های مختلف از دیدگاه‌های عامل، دانش و نوع خروجی بررسی و مقایسه شده است. در فصل چهارم نیز به جمع‌بندی، نتیجه‌گیری از مطالب آورده شده در گزارش و کارهای آینده پرداخته‌ایم.

¹⁹ Agent-based Systems

فصل ۲

تعاریف و مفاهیم مبنایی

۱-۲ مقدمه

برای درک عمیق روش‌های نوین ارائه شده در این گزارش، ضروری است که با مفاهیم و تعاریف پایه‌ای در حوزه‌های یادگیری ماشین خودکار، مدل‌های زبانی بزرگ و سیستم‌های عامل - محور آشنا شویم. این فصل به مرور این مبانی نظری اختصاص دارد و به عنوان سنگ بنایی برای تحلیل‌های ارائه شده در فصل سوم عمل خواهد کرد.

۲-۲ یادگیری خودکار ماشین

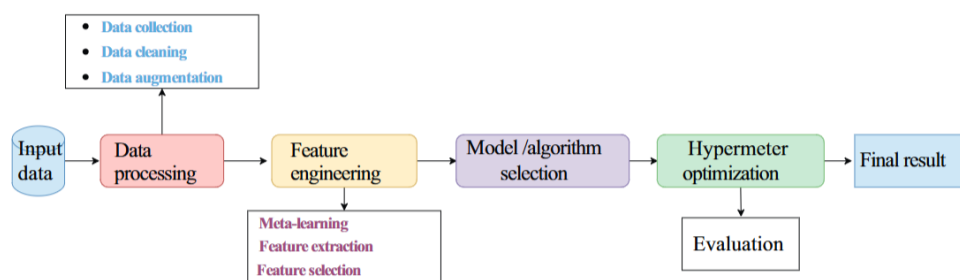
یادگیری ماشین خودکار به فرایند خودکارسازی وظایف تکراری و تخصصی در طراحی و استقرار مدل‌های یادگیری ماشین اطلاق می‌شود. هدف نهایی یادگیری ماشین خودکار، کاهش نیاز به دخالت متخصصان انسانی و تسریع فرایند کشف و اعتبارسنجی مدل‌های کارآمد است. این فرایند معمولاً شامل مراحل پیش‌پردازش داده^۱، مهندسی ویژگی، انتخاب مدل و بهینه‌سازی ابرپارامتر می‌باشد. همانطور که در شکل ۲-۱ نشان داده شده است، چارچوب یادگیری ماشین خودکار شامل مراحل کلیدی از پیش‌پردازش داده تا استقرار

^۱data preprocessing

جدول ۲-۱: مراحل و وظایف کلیدی در چارچوب یادگیری ماشین خودکار

مرحله	وظایف	تکنیک‌های خودکارسازی	خروجی
۱	پیش‌پردازش داده	رسیدگی به داده‌های گمشده، نرمال‌سازی، کدگذاری متغیرهای دسته‌ای	داده‌های آماده آموزش
۲	مهندسی ویژگی	انتخاب ویژگی، استخراج ویژگی، تبدیل ویژگی	مجموعه ویژگی‌های بهینه
۳	انتخاب مدل و بهینه‌سازی ابرپارامتر	مسئله CASH، جستجوی شبکه‌ای، بهینه‌سازی بیزی	مدل و ابرپارامترهای بهینه
۴	آموزش و ارزیابی	اعتبارسنجی متقابل، توقف زود هنگام، روش‌های ترکیبی	مدل نهایی آموزش دیده
۵	استقرار مدل	فشرده‌سازی مدل، بهینه‌سازی استنتاج	مدل آماده تولید

مدل است. هر مرحله می‌تواند با استفاده از الگوریتم‌ها و تکنیک‌های مختلف خودکارسازی شود تا بهینه‌ترین مدل برای یک مسئله خاص یادگیری ماشین شناسایی گردد. در جدول ۲-۱، مراحل و وظایف کلیدی در چارچوب یادگیری ماشین خودکار به همراه تکنیک‌های معمول خودکارسازی و خروجی‌های مرتبط ارائه شده است.



شکل ۲-۱: چارچوب یادگیری ماشین خودکار [۱].

مسئله CASH^۲: صورت بندی و تفسیر یکپارچه.

در یادگیری ماشین خودکار، مسئله CASH به دنبال یافتن زوج (A^*, λ^*) است که با انتخاب هم زمان نوع الگوریتم و پیکربندی ابرپارامترهای آن، کمترین خطا را روی داده های اعتبارسنجی (مثلاً در اعتبارسنجی متقابل k -بخشی) حاصل کند [۲]:

$$A^*, \lambda^* \in \arg \min_{A^{(j)} \in \mathcal{A}, \lambda \in \Lambda^{(j)}} \frac{1}{k} \sum_{i=1}^k \mathcal{L}(A_{\lambda}^{(j)}, D_{\text{train}}^{(i)}, D_{\text{valid}}^{(i)}). \quad (1-2)$$

که در آن $\mathcal{A}$ مجموعه الگوریتم های کاندید است و برای هر $A^{(j)} \in \mathcal{A}$ فضای ابرپارامتر $\Lambda^{(j)}$ تعریف می شود؛ $A_{\lambda}^{(j)}$ نمونه سازی الگوریتم j با پیکربندی $\lambda \in \Lambda^{(j)}$ است؛ $D_{\text{train}}^{(i)}$ و $D_{\text{valid}}^{(i)}$ به ترتیب داده های آموزش و اعتبارسنجی در تکرار i از اعتبارسنجی متقابل k -بخشی اند؛ k تعداد بخش هایی است که خطا روی آن ها میانگین گیری می شود؛ و $\mathcal{L}(\cdot)$ تابع زیانی است که باید کمینه شود (برای معیارهای حداکثری مانند دقت یا AUC می توان از منفی امتیاز به عنوان زیان استفاده کرد). بدین ترتیب، جست و جو در CASH هم زمان روی فضای انتخاب الگوریتم و فضای ابرپارامترهای هر الگوریتم انجام می شود تا زوج بهینه (A^*, λ^*) با بهترین عملکرد اعتبارسنجی به دست آید.

۲-۲-۱ بهینه سازی ابرپارامتر

ابرپارامترها^۳ پارامترهایی در مدل یادگیری ماشین هستند که مقدار آن ها پیش از آغاز فرایند آموزش تنظیم می شود (برخلاف پارامترها یا وزن ها که در طول آموزش یادگرفته می شوند). بهینه سازی ابرپارامتر فرایند یافتن ترکیبی از ابرپارامترها است که منجر به بهترین کارایی^۴ مدل بر روی یک مجموعه داده اعتبارسنجی می شود. روش های متداول برای این کار شامل جستجوی شبکه ای^۵، جستجوی تصادفی^۶ و بهینه سازی بیزی است. این فرایند به دلیل هزینه

^۲Combined Algorithm Selection and Hyperparameter Optimization

^۳Hyperparameters

^۴performance

^۵Grid Search

^۶Random Search

محاسباتی^۷ بالای ارزیابی هر پیکربندی^۸، بسیار چالش برانگیز است. در جدول ۲-۲، مقایسه‌ای از روش‌های مختلف بهینه‌سازی ابرپارامتر ارائه شده است.

جدول ۲-۲: مقایسه روش‌های بهینه‌سازی ابرپارامتر

روش	مزایا	معایب	کاربرد
جستجوی شبکه‌ای	پوشش کامل فضای گسسته‌شده، پیاده‌سازی ساده	هزینه بسیار مقیاس‌پذیری ضعیف	فضاهای ابرپارامتر کوچک
جستجوی تصادفی	کارآمدتر از جستجوی شبکه‌ای، مقیاس‌پذیری بهتر	عدم تضمین یافتن بهینه سراسری	فضاهای ابرپارامتر متوسط تا بزرگ
بهینه‌سازی بیزی	کارآمد در تعداد ارزیابی کم، استفاده از اطلاعات قبلی	پیچیدگی پیاده‌سازی، هزینه محاسباتی مدل جانشین	فضاهای پیوسته و ارزیابی گران‌قیمت

$$\lambda^* = \arg \min_{\lambda} \mathcal{L}_V^*(\lambda) = \arg \min_{\lambda} \mathcal{L}_V(\lambda, \mathbf{w}^*(\lambda)), \quad (2-2)$$

$$\text{s.t. } \mathbf{w}^*(\lambda) = \arg \min_{\mathbf{w}} \mathcal{L}_T(\lambda, \mathbf{w}). \quad (3-2)$$

در معادله (۲-۲)، هدف کمینه‌کردن زیان اعتبارسنجی با پارامترهای پاسخ بهینه $\mathcal{L}_V^*$ است؛ این کمینه‌سازی زمانی معنا دارد که معادله (۳-۲) تا همگرایی حل شده باشد. در این فرمول‌ها، $\mathcal{L}_V$ و $\mathcal{L}_T$ به ترتیب اهداف آموزش و اعتبارسنجی هستند و λ و $\mathbf{w}$ نیز به ترتیب ابرپارامترها و پارامترهای مدل‌اند. بهینه‌سازی ابرپارامتر می‌تواند به صورت ترتیبی انجام شود؛ بدین معنا که پیشنهاد λ_n به دنباله مقادیر قبلی $\{\lambda_1, \lambda_2, \dots, \lambda_{n-1}\}$ و زیان‌های اعتبارسنجی متناظرشان وابسته است.

⁷computational cost

⁸configuration

۲-۲-۲ جستجوی معماری شبکه عصبی

جستجوی معماری عصبی یکی از زیرشاخه های یادگیری ماشین خودکار است که به طور خاص بر خودکارسازی طراحی معماری شبکه های عصبی عمیق^۹ تمرکز دارد. به جای تکیه بر شهود^{۱۰} و طراحی دستی توسط متخصصان، جستجوی معماری عصبی از الگوریتم های جستجو^{۱۱} (مانند یادگیری تقویتی^{۱۲} یا روش های تکاملی^{۱۳}) برای کاوش در فضای طراحی^{۱۴} وسیع معماری ها استفاده می کند. هدف، یافتن معماری ای است که بهترین توازن را میان دقت و کارایی محاسباتی^{۱۵} برقرار کند.

۲-۳ مدل های زبانی بزرگ

مدل های زبانی بزرگ مدل های زبانی هستند که با استفاده از معماری ترنسفورمر و بر روی حجم عظیمی از داده های متنی آموزش دیده اند. این مدل ها دارای میلیاردها پارامتر هستند و توانایی های شگفت انگیزی در درک بافتار^{۱۶}، تولید متن منسجم^{۱۷} و استدلال از خود نشان می دهند.

۲-۳-۱ تاریخچه

توسعه مدل های زبانی بزرگ با معرفی معماری ترنسفورمر [۳] شتاب گرفت. مدل هایی مانند BERT [۴] و GPT [۵] چارچوب نظری^{۱۸} پیش آموزش^{۱۹} و ریزتنظیم^{۲۰} را تثبیت کردند.

^۹Deep Neural Networks^{۱۰}intuition^{۱۱}search algorithms^{۱۲}Reinforcement Learning^{۱۳}evolutionary methods^{۱۴}design space^{۱۵}computational efficiency^{۱۶}understanding context^{۱۷}coherent^{۱۸}paradigm^{۱۹}pre-training^{۲۰}fine-tuning

نسل های بعدی این مدل ها، با افزایش چشمگیر مقیاس^{۲۱} داده و پارامترها، به توانایی های یادگیری بافتاری^{۲۲} دست یافتند که به آن ها اجازه می دهد وظایف جدید را بدون ریزتنظیم و تنها با دیدن چند مثال در دستور^{۲۳} انجام دهند.

۲-۳-۲ معماری

پایه و اساس اکثر مدل های زبانی بزرگ مدرن، معماری ترنسفورمر است که بر مکانیزم خود توجه^{۲۴} تکیه دارد. این مکانیزم به مدل اجازه می دهد تا وابستگی های دوربرد^{۲۵} را در متن مدل کند و به بخش های مختلف ورودی وزن های متفاوتی اختصاص دهد. همانطور که در شکل ۲-۲ نشان داده شده است، مدل ها معمولاً از پشته ای از لایه های کدگذار^{۲۶} (مانند BERT) یا کدگشا^{۲۷} (مانند GPT) یا هر دو (مانند T5) تشکیل شده اند [۳].

۳-۳-۲ کاربردها

مدل های زبانی بزرگ کاربردهای متنوعی از جمله ترجمه ماشینی^{۲۸} [۶]، خلاصه سازی متن^{۲۹}، پرسش و پاسخ و تولید محتوا دارند. اخیراً، توانایی آن ها در تولید کد^{۳۰} [۷] و حل مسائل منطقی [۸]، درهای جدیدی را برای استفاده از آن ها در حوزه های فنی تر مانند مهندسی نرم افزار و یادگیری ماشین خودکار گشوده است.

²¹scale

²²In-Context Learning (ICL)

²³prompt

²⁴self-attention

²⁵long-range dependencies

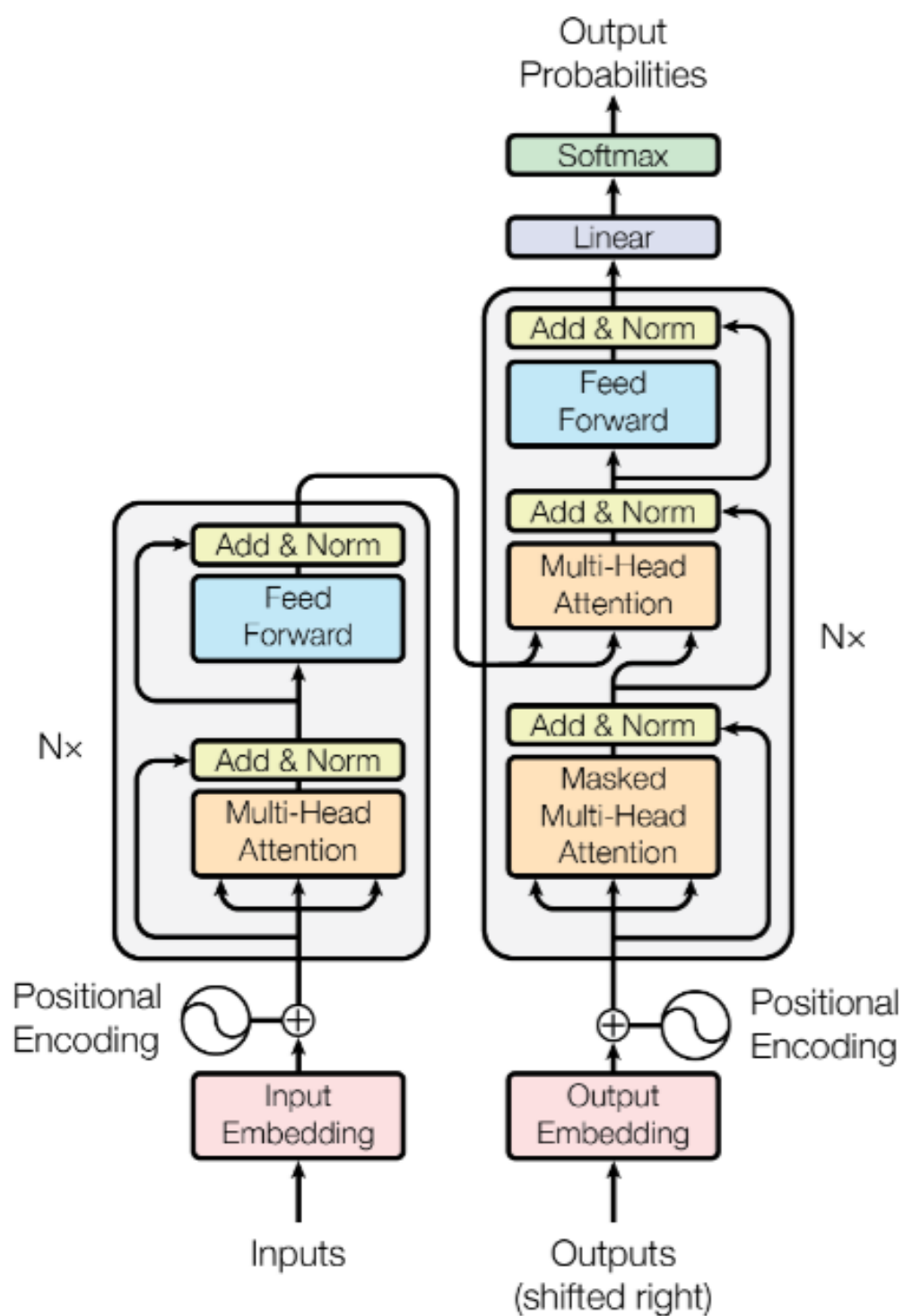
²⁶encoder

²⁷decoder

²⁸machine translation

²⁹text summarization

³⁰code generation



شکل ۲-۲: معماری ترنسفورمر [۳].

۴-۳-۲ تفکر و عمل

فراتر از تولید پاسخ‌های ایستا^{۳۱}، مدل‌های زبانی بزرگ می‌توانند برای تفکر^{۳۲} و عمل^{۳۳} نیز به کار روند. چارچوب‌هایی مانند ReAct [۹] نشان دادند که چگونه یک مدل زبانی بزرگ می‌تواند به صورت درهم‌تنیده^{۳۴}، ردپاهای استدلالی^{۳۵} (تفکر) و اقدامات (مانند جستجو در وب یا اجرای دستور) تولید کند. این قابلیت، سنگ بنای استفاده از مدل‌های زبانی بزرگ به عنوان هسته تصمیم‌گیرنده در عامل‌های خودمختار است. شکل ۲-۳ نمونه‌ای از این چارچوب را نشان می‌دهد.

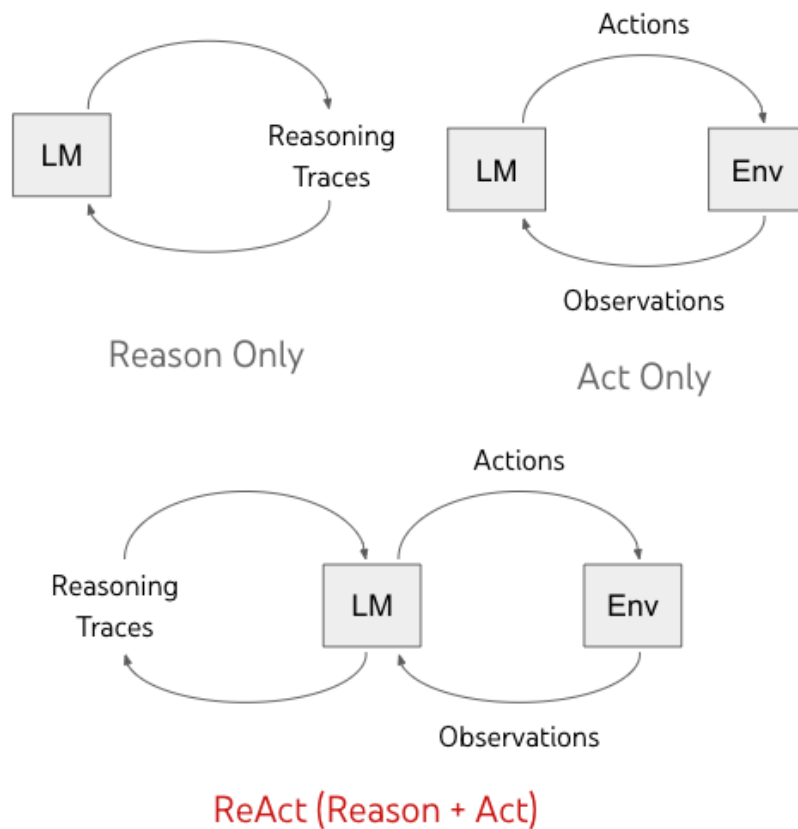
۴-۲ عامل

در زمینه هوش مصنوعی، عامل به سیستمی اطلاق می‌شود که در یک محیط^{۳۶} قرار دارد، آن را از طریق حسگرها^{۳۷} ادراک می‌کند و از طریق کنشگرها^{۳۸} بر آن تأثیر می‌گذارد تا به اهداف خود دست یابد. عامل‌های زبانی نوع خاصی از عامل‌ها هستند که از مدل‌های زبانی بزرگ به عنوان موتور استدلال اصلی خود برای پردازش ادراکات (اغلب متنی)، برنامه‌ریزی^{۳۹} و انتخاب اقدام استفاده می‌کنند [۱۰].

۴-۴-۱ تک‌عاملی^{۴۰}

یک سیستم شامل یک عامل واحد است که تمام وظایف ادراک، استدلال و عمل را به تنهایی انجام می‌دهد. در زمینه یادگیری ماشین خودکار، این می‌تواند یک عامل مبتنی بر مدل زبانی بزرگ باشد که کل خط لوله بهینه‌سازی را از ابتدا تا انتها مدیریت می‌کند [۱۰].

³¹static responses³²Reasoning³³Action³⁴interleaved³⁵reasoning traces³⁶environment³⁷sensors³⁸actuators³⁹planning⁴⁰Single-Agent



شکل ۲-۳: چارچوب ReAct [۹].

۲-۴-۲ چندعاملی^{۴۱}

سیستم‌های چندعاملی شامل دو یا چند عامل هستند که در یک محیط مشترک با یکدیگر تعامل^{۴۲} می‌کنند. این تعامل می‌تواند همکارانه^{۴۳} (برای دستیابی به یک هدف مشترک)، رقابتی^{۴۴}، یا ترکیبی از هر دو باشد. در مسائل پیچیده، بهره‌گیری از چند عامل امکان تفکیک وظایف^{۴۵}، موازی‌سازی، و افزایش پایداری در برابر خطا را فراهم می‌کند. برای

⁴¹ Multi-Agent

⁴² interact

⁴³ collaborative

⁴⁴ competitive

⁴⁵ task decomposition

نمونه، یک عامل متخصص تحلیل داده، یک عامل مسئول برنامه ریزی/تولید راه حل^{۴۶}، و یک عامل منتقد^{۴۷} وظیفه ارزیابی خروجی و ارائه بازخورد را بر عهده می گیرند؛ گاه یک عامل هماهنگ کننده^{۴۸} نیز برای زمان بندی و حل تعارض ها^{۴۹} به کار می رود. هماهنگی میان عامل ها می تواند متمرکز یا غیرمتمرکز باشد و معمولاً از طریق پیام رسانی، اشتراک حافظه، یا پروتکل های ارتباطی^{۵۰} انجام می شود. از چالش های کلیدی می توان به تعریف اهداف/پاداش ها، تخصیص منابع، و مدیریت ناهمگونی مهارت ها اشاره کرد [۱۰].

۲-۵ تولید تقویت شده با بازیابی^{۵۱}

تولید تقویت شده با بازیابی [۱۱] روشی است که مدل های زبانی بزرگ را با یک سازوکار بازیابی اطلاعات^{۵۲} خارجی ترکیب می کند. به جای تکیه صرف بر دانش پارامتری (ذخیره شده در وزن های مدل)، تولید تقویت شده با بازیابی ابتدا اطلاعات مرتبط را از یک مجموعه متون^{۵۳} یا پایگاه دانش^{۵۴} بازیابی می کند و سپس این اطلاعات را به مدل زبانی بزرگ می دهد تا پاسخ نهایی را بر اساس آن تولید کند. این روش به کاهش توهم^{۵۵} و افزایش دقت و به روز بودن^{۵۶} اطلاعات کمک می کند [۱۲]. در شکل ۲-۴، چارچوب کلی تولید تقویت شده با بازیابی نشان داده شده است.

⁴⁶planning/solution generation

⁴⁷critic

⁴⁸coordinator

⁴⁹conflict resolution

⁵⁰communication protocols

⁵¹Retrieval-Augmented Generation (RAG)

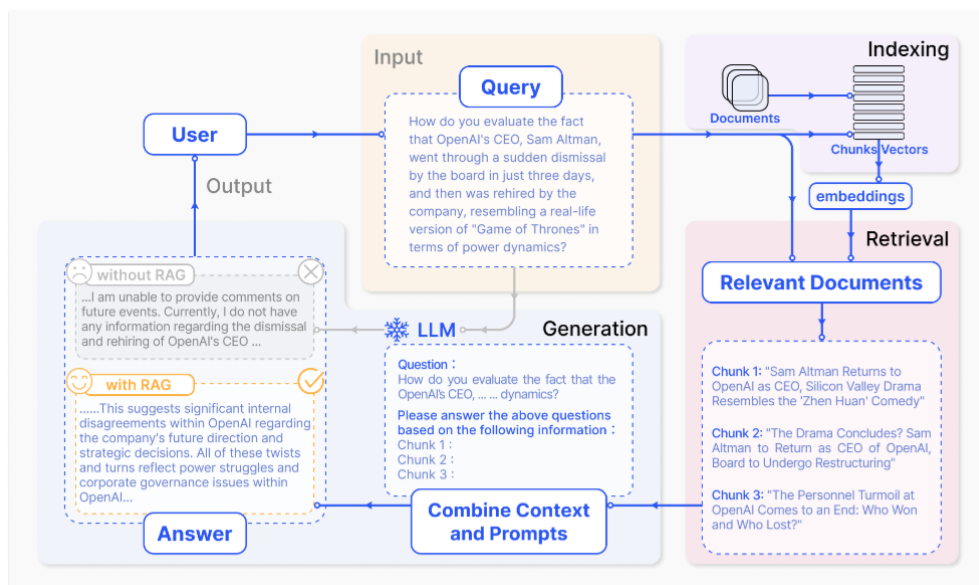
⁵²information retrieval mechanism

⁵³corpus

⁵⁴knowledge base

⁵⁵hallucination

⁵⁶up-to-dateness



شکل ۲-۴: چارچوب کلی تولید تقویت‌شده با بازیابی. این فرآیند شامل سه مرحله اصلی است: (۱) نمایه‌سازی، که در آن اسناد به قطعات برداری تبدیل و ذخیره می‌شوند؛ (۲) بازیابی، که در آن مرتبط‌ترین قطعات بر اساس شباهت معنایی با پرسش استخراج می‌شوند؛ و (۳) تولید، که در آن مدل زبانی بزرگ با استفاده از پرسش و قطعات بازیابی‌شده، پاسخ نهایی را تولید می‌کند [۱۳].

۲-۵-۱ پایگاه دانش

پایگاه دانش در تولید تقویت‌شده با بازیابی معمولاً مجموعه‌ای از اسناد^{۵۷} است. این اسناد اغلب به قطعات^{۵۸} کوچکتر تقسیم شده و به صورت نمایش‌های برداری^{۵۹} (یا جانمایی‌ها^{۶۰}) در یک پایگاه داده برداری^{۶۱} ذخیره می‌شوند تا بازیابی مبتنی بر شباهت معنایی^{۶۲} به سرعت انجام شود.

⁵⁷documents

⁵⁸chunks

⁵⁹vector representations

⁶⁰embeddings

⁶¹vector database

⁶²semantic similarity retrieval

۲-۵-۲ ترکیب با عامل

عامل‌های هوشمند می‌توانند از تولید تقویت شده با بازیابی به عنوان یک ابزار^{۶۳} کلیدی استفاده کنند. زمانی که یک عامل یادگیری ماشین خودکار با یک مجموعه داده جدید روبرو می‌شود، می‌تواند از تولید تقویت شده با بازیابی برای جستجو در پایگاه دانش استفاده کند. این دانش بازیابی شده به عامل کمک می‌کند تا تصمیمات آگاهانه‌تری در مورد مسئله اتخاذ کند [۱۴].

⁶³tool

فصل ۳

مروری بر کارهای مرتبط

۱-۳ مقدمه

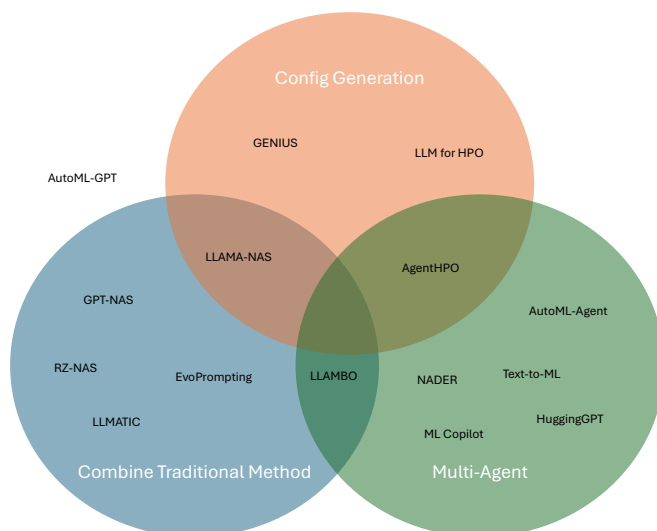
ادغام مدل‌های زبانی بزرگ در خودکارسازی یادگیری ماشین به عنوان رویکردی دگرگون‌ساز برای بهینه‌سازی معماری‌های عصبی و فرایارامترها پدیدار شده است. این فصل کارهای موجود را از سه منظر مکمل بررسی می‌کند: معماری عامل، راهبردهای تقویت دانش^۱، و طراحی قالب خروجی. این ابعاد در کنار هم نشان می‌دهند که سامانه‌های گوناگون چگونه از قابلیت‌های مدل‌های زبانی بزرگ بهره می‌برند و هم‌زمان محدودیت‌های ذاتی آن‌ها را مدیریت می‌کنند. در شکل ۳-۱، دسته‌بندی مقالات مرور شده بر مبنای این سه منظر ارائه شده است. بخش‌های بعدی به تفصیل هر یک از این ابعاد را بررسی می‌کنند و نمونه‌های شاخص را برای روشن‌ساختن رویکردهای مختلف ارائه می‌دهند.

۲-۳ ظهور نخستین روش‌های یادگیری ماشین خودکار

خودکارسازی یادگیری ماشین با هدف مردمی‌سازی فرایندهای پرهزینه و زمان‌بر گزینش مدل و بهینه‌سازی فرایارامتر پدید آمد. ریشه گزینش الگوریتم به سال ۱۹۷۶ بازمی‌گردد

¹knowledge augmentation strategies

فصل ۳. مروری بر کارهای مرتبط ۲-۳. ظهور نخستین روش‌های یادگیری ماشین خودکار



شکل ۳-۱: دسته بندی مقالات بر مبنای معماری عامل، استفاده از روش های سنتی، و قالب خروجی مدل زبانی بزرگ.

که در آن، مسئله انتخاب الگوریتم بهینه از میان یک مجموعه از پیش تعریف شده به منظور کمینه سازی افت کارایی روی داده های اعتبارسنجی صورت بندی شد [۱۵]. کوشش های نخستین در حوزه بهینه سازی فرایارامتر بر روش های منفردی چون جست وجوی شبکه ای و جست وجوی تصادفی متمرکز بود؛ روش هایی ساده اما ناکارا در فضا های بعد بالا [۱۶]. این روش نشان داد که جست وجوی تصادفی با تخصیص مؤثرتر منابع به فرایارامترهای اثرگذار، از جست وجوی شبکه ای پیشی می گیرد و بدین ترتیب زمینه را برای روش های پیشرفته تر فراهم ساخت [۱۶].

نقطه عطف در ۲۰۱۳ با معرفی مسئله CASH رخ داد؛ چارچوبی که گزینش الگوریتم و بهینه سازی فرایارامتر را در یک بهینه سازی واحد ادغام می کرد [۲]. بر مبنای این ایده، Auto-WEKA به عنوان نخستین سامانه جامع یادگیری ماشین خودکار پدید آمد و با اتکا به SMAC رویکردی مبتنی بر بهینه سازی بیزی که از جنگل های تصادفی^۲ به عنوان جانشین^۳ استفاده می کرد به جست وجو در میان رده بندهای WEKA و فرایارامترهایشان پرداخت [۱۷، ۲]. در ادامه، auto-sklearn در ۲۰۱۵ روش CASH را به الگوریتم های scikit-

^۲random forests

^۳surrogate

فصل ۳. مروری بر کارهای مرتبط ۲-۳. ظهور نخستین روش‌های یادگیری ماشین خودکار

learn بسط داد و با به‌کارگیری فرا-یادگیری^۴ برای شروع تازه و نیز ساخت مجمع^۵‌های مقاوم، بهبودهای معناداری رقم زد [۱۸].

به‌طور موازی، الگوریتم‌های تکاملی به‌عنوان بدیلی توانا تر مطرح شدند؛ چنان‌که TPOT خطوط لوله یادگیری ماشین را به‌صورت برنامه‌هایی درخت ساختار مدل می‌کند و با برنامه‌نویسی ژنتیکی^۶، ترکیب‌های انعطاف‌پذیری از پیش‌پردازنده‌ها، گزیننده‌های ویژگی^۷ و مدل‌ها را می‌آزماید [۱۹]. روش‌های چندوفایی^۸ با تخصیص تدریجی منابع به پیکربندی‌های امیدبخش و با الهام از راهبردهای باندیتی^۹ هزینه محاسباتی را مهار کردند [۲۰، ۲۱]. تمرکز این رویکردهای اولیه عمدتاً بر مسائل کلاسیک دسته‌بندی^{۱۰} و رگرسیون بود؛ و معیارهایی چون OpenML-CC18 ارزیابی‌های استاندارد شده را تسهیل کردند [۲۲].

گذار به جست‌وجوی معماری شبکه‌های عصبی از حوالی ۲۰۱۷ آغاز شد و طراحی معماری را امتداد طبیعی بهینه‌سازی فرایارامتر تلقی کرد. کارهای اولیه مبتنی بر یادگیری تقویتی در جست‌وجوی معماری شبکه‌های عصبی، گرچه پرهزینه، نقطه شروعی اثرگذار بودند و گونه‌های کاراتری همچون ENAS با اشتراک پارامتر^{۱۱} را الهام بخشیدند [۲۳، ۲۴]. در ادامه، معیارهایی مانند NAS-Bench-101 با فراهم‌کردن ارزیابی‌های ازپیش محاسبه‌شده، امکان پژوهش بازتولیدپذیر را مهیا کردند [۲۵]. این سیر تحول، تنظیم دستی را به خطوط لوله خودکار بدل کرد و بستر را برای همگرایی با روش‌های پیشرفته‌تر فراهم ساخت.

⁴meta-learning

⁵ensemble

⁶genetic programming

⁷feature selectors

⁸multi-fidelity

⁹bandit-based strategies

¹⁰classification

¹¹parameter sharing

۳-۳ تحلیل بر مبنای معماری عامل

۱-۳-۳ سامانه‌های تک‌عاملی

بیشینه سامانه‌های مبتنی بر مدل‌های زبانی بزرگ برای خودکارسازی یادگیری ماشین، معماری‌های تک‌عاملی را برمی‌گزینند که در آن یک عامل منفرد کل جریان بهینه‌سازی را مدیریت می‌کند. بر مبنای چگونگی ادغام مدل زبانی در فرایند بهینه‌سازی، این سامانه‌ها در چند روش عملیاتی متمایز قرار می‌گیرند.

بهینه‌سازی مستقیم از طریق دستوردهی تکراری^{۱۲}

در این رویکرد برجسته، مدل‌های زبانی به‌منزله بهینه‌سازهای جعبه‌سیاه به کار می‌روند که پیکربندی‌ها را پیشنهاد می‌کنند و از راه حلقه‌های بازخورد^{۱۳} آن‌ها را پالایش می‌کنند. مدل با تکیه بر تاریخچه آزمون‌ها^{۱۴} که به صورت گفت‌وگوهای چت محور یا خلاصه‌های فشرده انباشته می‌شود، بافتار^{۱۵} را حفظ می‌کند و پالایش تکراری مبتنی بر معیارهای اعتبارسنجی را ممکن می‌سازد (شکل ۲-۳) [۲۶، ۲۷]. این روش به چارچوب‌های بهینه‌سازی بیزی نیز بسط یافته است؛ جایی که مدل‌های آغاز گرم، نمونه‌گیری از نامزدها و مدل‌سازی جانشین^{۱۶} را با اتکاء به استدلال زبان طبیعی و مشروط بر تاریخچه بهینه‌سازی انجام می‌دهند [۲۸]. این رویکردها در تنظیمات با بودجه کم کارایی رقابتی نشان می‌دهند و بی‌آن‌که به ریزتنظیم نیاز داشته باشند، بر یادگیری بافتاری از توصیف مسئله و بازخورد تجربی تکیه می‌کنند.

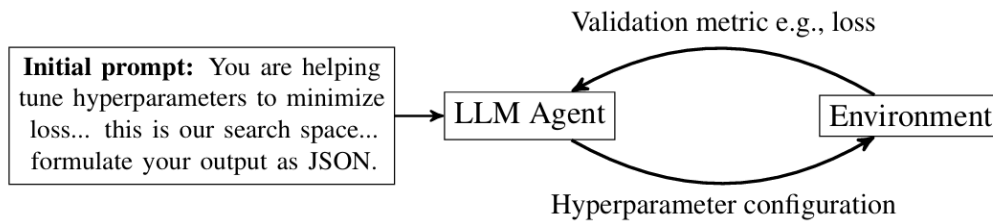
¹²iterative prompting

¹³feedback loops

¹⁴trials

¹⁵context

¹⁶surrogate modeling



شکل ۳-۲: مدل‌های زبانی بزرگ برای بهینه‌سازی ابرپارامترها. در این چارچوب تصمیم‌گیری ترتیبی، توصیف مسئله و فضای جست‌وجو را به‌عنوان دستور به مدل زبانی داده می‌شود. سپس مدل زبانی مجموعه‌ای از ابرپارامترها را برای ارزیابی پیشنهاد می‌کند. محیط یک اجرای آموزش را با این تنظیم ابرپارامتر اجرا می‌کند و سپس مقدار یک معیار اعتبارسنجی دوباره به‌عنوان دستور به مدل زبانی داده می‌شود [۲۶].

عملگرهای تکاملی^{۱۷}

راهبردی دیگر، مدل زبانی را به‌عنوان عملگرهای جهش^{۱۸} یا ترکیب^{۱۹} درون چارچوب‌های تکاملی جا می‌دهد. به‌جای جایگزینی الگوریتم‌های جست‌وجوی سنتی، این سامانه‌ها آن‌ها را با تنوع‌های تولیدشده توسط مدل زبانی تقویت می‌کنند. مدل می‌تواند تغییرات معماری مبتنی بر کد را در چارچوب‌های کیفیت-تنوع^{۲۰} بسازد (شکل ۳-۳) [۲۹] یا به‌صورت عملگرهای تطبیقی که میان نسل‌ها با تنظیم دستور^{۲۱} پالایش می‌شوند عمل کند [۳۰]. برخی پیاده‌سازی‌ها توانایی‌های بازتابی^{۲۲} را نیز می‌گنجانند؛ به این معنا که مدل پیامدهای جهش را تحلیل می‌کند و بازخورد زبانی^{۲۳} برای هدایت تکرارهای بعدی تولید می‌کند [۳۱]. این ادغام، استحکام^{۲۴} جست‌وجوی تکاملی را حفظ می‌کند و در عین حال از خلاقیت مدل در تولید تنوع‌های معنادار بهره می‌گیرد. نمونه‌ای شاخص، GPT-NAS است که در آن مدل زبانی به‌منزله یک بازساز^{۲۵} معماری عمل کرده و با ماسک‌گذاری و بازتولید لایه‌ها،

¹⁷Evolutionary Operators

¹⁸mutation

¹⁹crossover

²⁰quality-diversity

²¹prompt-tuning

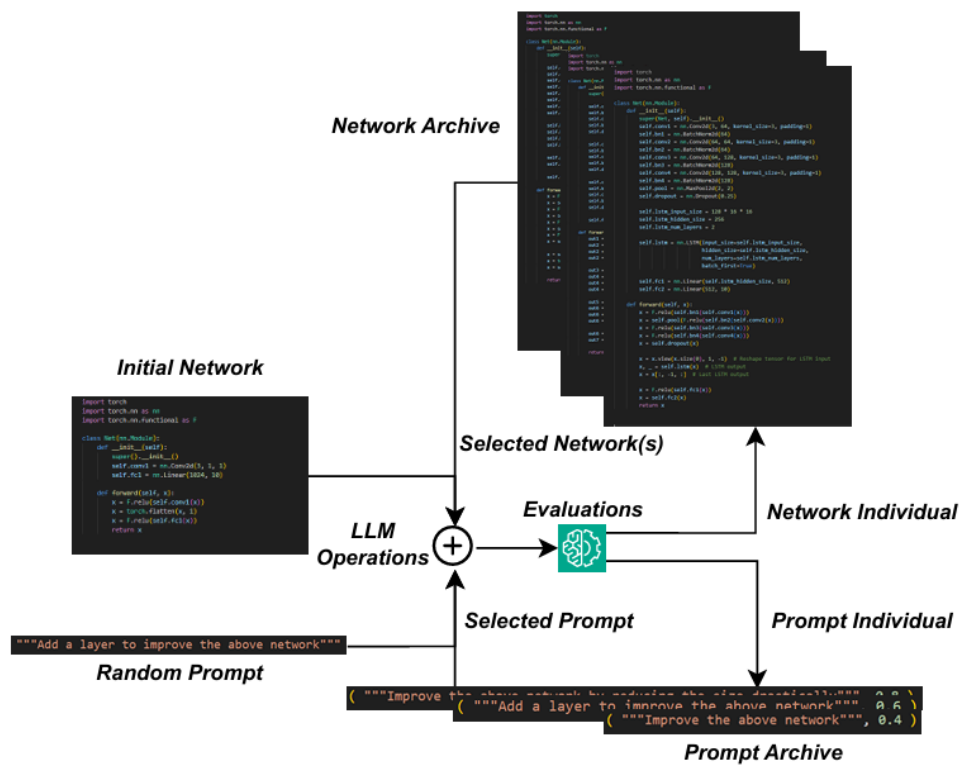
²²reflective capabilities

²³linguistic feedback

²⁴robustness

²⁵reconstructor

نامزدهای نمونه برداری شده توسط الگوریتم ژنتیک^{۲۶} را بهبود می دهد؛ بنابراین عملاً نقشی هم ارز با یک عملگر جهش آگاه از زمینه ایفا می کند بی آنکه راهبرد جست و جوی تکاملی را جایگزین کند [۳۲].



شکل ۳-۳: در این شکل، روند روش تکاملی نمایش داده شده است. در دور اولیه تکامل، یک شبکه اولیه با یک دستور تصادفی تحت یک عملیات جهش قرار می گیرد. سپس فرد شبکه و فرد دستور ارزیابی شده و در آرشیوهای جداگانه ذخیره می شوند. در طول حلقه تکاملی، دستور و شبکه انتخاب شده تحت یک عملیات تکاملی قرار می گیرند (در صورت استفاده از عملگر ترکیب، دستور ثابت باقی می ماند) تا شبکه ها و دستورهای بیشتری برای پر کردن و روشن سازی آرشیوها ایجاد شوند [۲۹].

²⁶genetic algorithm

کنترل‌گرهای جریان کار

در این روش، مدل‌های زبانی به‌مثابه هم‌آواساز^{۲۷} برای مدیریت اجزای خط لوله به کار می‌روند. سامانه با ترکیب دستورهای که فراداده ساختاریافته^{۲۸} - از جمله کارت‌های داده^{۲۹} و کارت‌های مدل^{۳۰} - را در خود دارند، مدل را در مراحل پیاپی از پردازش داده، انتخاب مدل تا تنظیم فرایارامتر هدایت می‌کند [۳۳، ۳۴]. برخی پیاده‌سازی‌ها برنامه‌های پیچیده یادگیری ماشین را به مولفه‌های ماژولار^{۳۱} تجزیه می‌کنند که به‌طور جداگانه تولید و با آزمون‌های واحد خودکار^{۳۲} راستی‌آزمایی می‌شوند تا سازگاری تضمین گردد [۳۵]. این رویکرد با شکستن خطوط لوله طولانی و ناهمگون به زیروظایف قابل مدیریت و اتکاء به دستوردهی زمینه‌مند^{۳۳}، انسجام کلی را حفظ می‌کند.

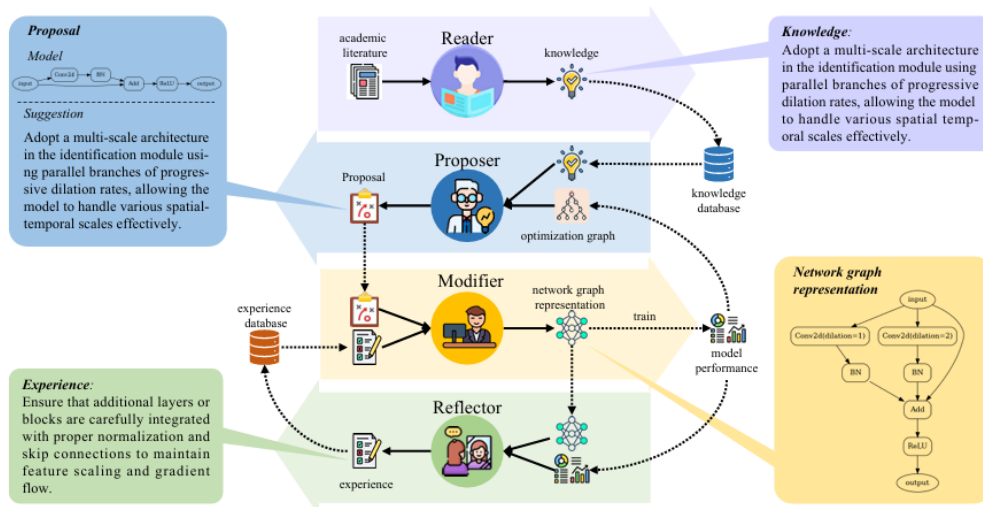
۳-۳-۲ سامانه‌های چندعاملی

معماری‌های چندعاملی با تفکیک نقش، زیروظایف یادگیری ماشین خودکار را میان عامل‌هایی با قابلیت‌های مکمل توزیع می‌کنند. این سامانه‌ها از رهگذر تفکیک کارکردی^{۳۴} و همکاری بین‌عاملی^{۳۵}، مدیریت پیچیدگی و استدلال پیچیده‌تر را ممکن می‌سازند.

همکاری مبتنی بر نقش^{۳۶}

الگوی پایه، دو عامل تخصصی با مسئولیت‌های متمایز را به کار می‌گیرد. در یک ساختار، تولید پیکربندی از اجرای تجربی جدا می‌شود: عامل سازنده نیازمندی‌ها را تفسیر و پیکربندی‌های پیشنهادی همراه با استدلال ارائه می‌کند و عامل اجراکننده آموزش را انجام داده و نتایج را در

²⁷orchestrator²⁸structured metadata²⁹data cards³⁰model cards³¹modular components³²automated unit tests³³contextual prompting³⁴functional decomposition³⁵inter-agent collaboration³⁶Role-Based Collaboration



شکل ۳-۴: نمای کلی از چارچوب مبتنی بر نقش. خواننده به‌طور مداوم از ادبیات دانشگاهی می‌آموزد، در حالی که پیشنهاددهنده امیدوارکننده‌ترین شبکه‌های نامزد را شناسایی کرده و اصلاحاتی را پیشنهاد می‌کند. اصلاح‌کننده این پیشنهادها را پیاده‌سازی می‌کند و بازتاب‌دهنده نتایج را تحلیل و بازخورد ارائه می‌دهد. عملکرد شبکه اصلاح‌شده به پیشنهاددهنده بازگردانده می‌شود تا پیشنهادهای بعدی را آگاه سازد و یک چرخه بهبود مستمر را تقویت کند [۳۶].

گزارش‌های مشترک می‌گنجاند تا چرخه‌های بعدی پیشنهادهای سازنده را تغذیه کند [۳۷]. این تقسیم کار بازتاب گردش کار متخصصان است و با حافظه ترتیبی^{۳۷} انباشته، به عملکرد خودگردان بدون مداخله انسانی می‌انجامد. افزون بر این، همین الگورا می‌توان به سطح تیمی تعمیم داد: نمونه پیشرفته، تقسیم عامل‌ها به تیم پژوهش^{۳۸} و تیم توسعه^{۳۹} است که به ترتیب دانش را از ادبیات پژوهشی^{۴۰} استخراج و پیشنهادهای تغییر را می‌سازند، و آن پیشنهادها را بر نمایش‌های گراف^{۴۱} اعمال کرده و هم بازخورد فوری و هم استخراج تجربه بلندمدت فراهم می‌کنند (شکل ۳-۴) [۳۶]. در این قالب، پایگاه‌های داده برداری^{۴۲} با بازیابی مبتنی بر شباهت^{۴۳} برای آمیختن دانش ادبیات با سوابق طراحی به کار می‌روند تا چرخه‌های بعدی

³⁷ episodic memory

³⁸ Research Team

³⁹ Development Team

⁴⁰ literature

⁴¹ graph representations

⁴² vector databases

⁴³ similarity-based retrieval

پیشنهاددهی و اجرا بهتر هدایت شوند.

هماهنگی سلسله‌مراتبی^{۴۴}

در ساختارهای پیچیده‌تر، چند عامل تخصصی تحت نظارت یک عامل مدیر^{۴۵} سازمان می‌یابند. مدیر با استدلال تقویت‌شده با بازیابی^{۴۶} طرح‌های متنوعی می‌سازد، آن‌ها را به زیروظایف قابل موازی‌سازی^{۴۷} واگشایی و به عامل مناسب تخصیص می‌دهد و از رهگذر راستی‌آزمایی چندمرحله‌ای و حلقه‌های بازنگری^{۴۸} نتایج را اعتبارسنجی می‌کند [۳۸].

۴-۳ تحلیل منابع دانش^{۴۹}

کارآمدی سامانه‌های خودکارسازی یادگیری ماشین مبتنی بر مدل‌های زبانی بزرگ به‌صورت بنیادین به چگونگی اکتساب، مدیریت و بهره‌برداری از دانش در سراسر فرایند بهینه‌سازی وابسته است. رویکردهای معاصر طیفی از راهبردهای تقویت دانش را پوشش می‌دهند؛ از یادگیری درون‌متنی صرف تا چارچوب‌های تولید تقویت‌شده با بازیابی که هر یک پیامدهای متمایزی برای کارایی جست‌وجو و تعمیم‌پذیری^{۵۰} دارند.

۳-۴-۱ دانش درونی: تاریخچه آزمون و بازتاب^{۵۱}

این رسته، دانش تولیدشده توسط خود سامانه را در بر می‌گیرد از اتکای مستقیم به تاریخچه آزمون‌ها در پنجره بافتار^{۵۲} تا حافظه رویدادی و خودبازتابی^{۵۳} که به‌تدریج به بازخورد قابل

⁴⁴ Hierarchical Coordination

⁴⁵ Agent Manager

⁴⁶ retrieval-augmented reasoning

⁴⁷ parallelizable subtasks

⁴⁸ revision loops

⁴⁹ Knowledge Source Analysis

⁵⁰ generalization

⁵¹ System-Internal Knowledge: Trials and Reflection

⁵² context window

⁵³ self-reflection

اقدام^{۵۴} و اصول طراحی تقطیر می‌شوند.

```
Epoch: [0, 5, 10, 15, 20]
Train Acc: [39.54, 82.49, 96.30, 97.73, 98.30]
Val Acc: [50.3, 77.76, 82.73, 81.34, 82.69]
Total Training Time: 1442.33s | Final Valid Acc: 82.43%
```

The model showed signs of overfitting, as indicated by the perfect training accuracy and the plateauing validation accuracy.

شکل ۳-۵: یادگیری درون‌متنی از تاریخچه بهینه‌سازی [۳۷].

یادگیری درون‌متنی از تاریخچه بهینه‌سازی^{۵۵}

یک راهبرد رایج، پنجره زمینه مدل را همچون مخزن اصلی دانش در نظر می‌گیرد و صرفاً بر تاریخچه آزمون‌های انباشته‌شده طی فرایند بهینه‌سازی^{۵۶} تکیه می‌کند. سامانه‌های مبتنی بر این ایده، پیکربندی‌های پیشین و سنجه‌های کارایی متناظر را در دستور می‌گنجانند تا مدل بتواند از رهگذر بازخورد تکرارشونده پیشنهادها را پالایش کند [۲۶، ۲۷، ۲۸]. این تاریخچه ممکن است به قالب‌های گوناگونی سریال‌سازی^{۵۷} شود: گفت‌وگوهای گفت‌وگوهای چت محور^{۵۸} که توالی زمانی تعاملات را حفظ می‌کنند، خلاصه‌های فشرده برای مهار قیود طول زمینه، الگوهای اندک‌نمونه^{۵۹} برگزیده از جمعیت ارزیابی‌شده [۲۶، ۳۰]، و نیز تجربه‌های تاریخی استانداردسازی‌شده که داده‌های ناهمگن گذشته (پیکربندی‌های JSON^{۶۰}، پیاده‌سازی‌های کد، سنجه‌های عددی) را به نمایش‌های یکنواخت زبان طبیعی تبدیل می‌کنند تا پردازش و قیاس توسط مدل تسهیل شود [۳۹]. شکل ۳-۵ این رویکردها را به‌مثابه طیفی از راهبردهای یادگیری درون‌متنی نشان می‌دهد که استفاده از تاریخچه بهینه‌سازی را شامل می‌شود [۳۷].

⁵⁴ actionable feedback

⁵⁵ In-Context Learning from Optimization History

⁵⁶ optimization

⁵⁷ serialization

⁵⁸ chat-style dialogues

⁵⁹ few-shot demonstrations

⁶⁰ JavaScript Object Notation

در این رویکرد تکمیلی، ابرپارامترهای عددی برای بهبود استدلال گسسته‌سازی^{۶۱} می‌شوند (مثلاً به سطوح «کم»، «متوسط»، «زیاد»)، و تجربه‌های استانداردسازی شده با جانمایی کردن^{۶۲} و نمایه‌سازی در پایگاه برداری پشتیبانی می‌شوند تا بازیابی مبتنی بر شباهت، چند برترین های نمونه مرتبط را برای نمایش درون‌متنی برگزینند. فراتر از درج خام تجربه‌ها، استخراج دانش برون‌خط^{۶۳} از طریق خلاصه‌سازی تکرارشونده مبتنی بر مدل و اعتبارسنجی پسین^{۶۴} روی وظایف کنارگذاشته، اصول طراحی سطح بالا و بازخورد قابل اقدام را تقطیر می‌کند؛ این دانش استخراج شده می‌تواند به صورت راهنمای سامانه یا الگوهای اندک نمونه در متن تزریق شود و حتی تولید راه حل تک نمونه‌ای^{۶۵} برای وظایف نو را میسر سازد [۳۹].

این روش به ویژه در سناریوهای کم بودجه مؤثر است؛ جایی که پیشین‌های یادگرفته شده مدل و تجربه‌های استانداردسازی شده می‌توانند بدون داده تجربی فراوان مسیر اکتشاف را هدایت کنند. در بسترهای بهینه‌سازی بیزی، مشاهدات تاریخی، آغاز گرم، نمونه برداری نامزد^{۶۶} و مدل‌سازی جانشین را به طور کامل از راه سریال سازی زبان طبیعی ارزیابی های پیشین شرطی سازی می‌کنند؛ و در حالی که در حالت های بی نمونه^{۶۷} یا کم نمونه^{۶۸} عمل می‌کنند، کارایی رقابتی در قیاس با روش های سنتی نشان می‌دهند [۲۸]. تجربه های استاندارد سازی شده با فراهم سازی نمونه های مشابه تأیید شده و قواعد طراحی تقطیر شده، می‌توانند این مراحل را دقیق تر شروع تازه^{۶۹} کنند و میدان جست و جو را به صورت هدایت شده منقبض سازند [۳۹].

محدودیت اصلی در بهینه‌سازی های بلندافق^{۷۰} رخ می‌نماید؛ جایی که قیود پنجره زمینه مستلزم نگهداشت گزینشی یا فشرده سازی با از دست رفتن اطلاعات تاریخیچه است و چه بسا الگوهای حیاتی برای پالایش مرحله پایانی را حذف کند. کانونی سازی^{۷۱} تا حدی این معضل را با خلاصه های ساخت یافته متراکم و بازیابی هدفمند تخفیف می‌دهد، اما همچنان با برش

⁶¹ discretized⁶² embed⁶³ offline knowledge elicitation⁶⁴ post-validation⁶⁵ one-shot solution generation⁶⁶ candidate sampling⁶⁷ zero-shot⁶⁸ few-shot⁶⁹ warmstart⁷⁰ long-horizon⁷¹ canonicalization

اطلاعاتی، سوگیری‌های ناشی از گسسته‌سازی، و حساسیت به امتیازدهی ارتباط^{۷۲} و پوشش مخزن مواجه است. با این‌همه، چون مصرف نهایی این دانش درون همان پنجره زمینه صورت می‌گیرد، مرز میان «دانش درون‌متنی صرف» و «تقویت مبتنی بر بازیابی» کم‌رنگ‌تر می‌شود؛ و ادغام تاریخچه آزمون با تجربه‌های استانداردسازی‌شده، پایایی و کارایی یادگیری درون‌متنی را در عمل ارتقا می‌دهد [۲۶، ۲۷، ۳۰، ۲۸، ۳۹].

۳-۴-۲ دانش بیرونی: بازیابی از ادبیات و مخازن^{۷۳}

سامانه‌های پیشرفته‌تر، تولید تقویت‌شده با بازیابی را برای ادغام دانش بیرون از مسیر بهینه‌سازی به کار می‌گیرند. این چارچوب‌ها مخازن دانش ساخت‌یافته عموماً پایگاه‌های داده برداری نمایه‌شده با نهفتارها نگه می‌دارند تا بر پایه زمینه وظیفه جاری، اطلاعات مرتبط را بازیابی کرده و در راهنماهای متنی تزریق کنند و بدین‌سان تصمیم‌سازی را اطلاع‌رسانی کنند.

استخراج دانش مبتنی بر ادبیات پژوهشی^{۷۴}

چند رویکرد، دانش راهبردی را از ادبیات علمی برای هدایت تصمیم‌های معماری گردآوری می‌کنند. یک راهبرد از کارگزاران خوانش تخصصی^{۷۵} بهره می‌برد که مقالات اخیر را خزش^{۷۶} کرده، نکته‌های روش‌شناختی را از چکیده‌ها و بخش‌های روش استخراج می‌کنند و در پایگاه‌های داده برداری برای بازیابی مبتنی بر شباهت بایگانی می‌نمایند [۳۶]. در خلال بهینه‌سازی، پیشنهادها تغییر به‌منزله پرسش، اصول طراحی مرتبط را فراخوانی می‌کنند؛ و بدین‌ترتیب سامانه بدون آن‌که مدل پایه الزاماً بر تازه‌ترین انتشارات آموزش دیده باشد، از مرز دانش روز بهره می‌گیرد. نمونه‌ای دیگر، خلاصه‌هایی از مقالات arXiv و جست‌وجوهای وب را از طریق رابط‌های برنامه‌نویسی کاربردی^{۷۷} بازیابی کرده و راهنماهای برنامه‌ریزی را

⁷²relevance scoring

⁷³External Knowledge via Retrieval

⁷⁴Literature-Driven Knowledge Extraction

⁷⁵specialized reader agents

⁷⁶crawl

⁷⁷application programming interfaces (APIs)

با بینش‌های بیرونی پیرامون مدل‌ها، ابرپارامترها و مجموعه داده‌ها غنی می‌کند تا تنوع و سازگاری طرح را ارتقا دهد [۳۸].

این استخراج دانش، برای طراحی معماری‌های عصبی بس سودمند است؛ چراکه نوآوری‌های اخیر در ترکیب لایه‌ها، اتصالات پرشی^{۷۸} یا شماهای نرمال‌سازی^{۷۹} چه‌بسا در پارامترهای منجمد^{۸۰} مدل بازتاب نیافته باشند. با این‌همه، کیفیت دانش بازیابی‌شده به‌نحو حساس به سازوکار امتیازدهی ارتباط و پوشش پیکره ادبیات نمایه‌شده وابسته است.

مخازن مجموعه داده و مدل^{۸۱}

در کنار بازیابی مبتنی بر ادبیات، چند سامانه از مخازن بیرونی برای فراداده‌های مجموعه داده‌ها و مدل‌های ازپیش‌آموزش‌دیده پرس‌وجو می‌کنند. چارچوب‌هایی که کل زنجیره خودکارسازی یادگیری ماشین را راهبری می‌کنند، کارت‌های مجموعه داده از پلتفرم‌هایی مانند Kaggle و کارت‌های مدل از HuggingFace را بازیابی کرده و فراداده ساخت‌یافته از جمله وجه‌های داده^{۸۲}، متغیرهای هدف، معماری‌های مدل و بازه‌های ابرپارامتر را در راهنماهای متنی می‌گنجانند تا تصمیم‌های پایین‌دستی را غنی کنند (شکل ۳-۶) [۳۸، ۳۴]. برای مجموعه داده‌های نادیده، سازوکارهای انتقال مبتنی بر شباهت^{۸۳} با محاسبه همبستگی میان کدگذاری کارت‌های داده (با مدل‌هایی مانند CLIP) مسائل مشابه را شناسایی کرده و ابرپارامترها یا الگوهای معماری را از تجربه‌های تاریخی منتقل می‌سازند [۳۳].

این تقویت مبتنی بر فراداده^{۸۴} تعمیم‌پذیری میان حوزه‌های گوناگون را بدون نیاز به آموزش‌های خاص وظیفه ممکن می‌کند؛ هرچند به دسترس‌پذیری مخازن خوش‌سامان و برچسب‌گذاری دقیق فراداده متکی است.

⁷⁸skip connections

⁷⁹normalization schemes

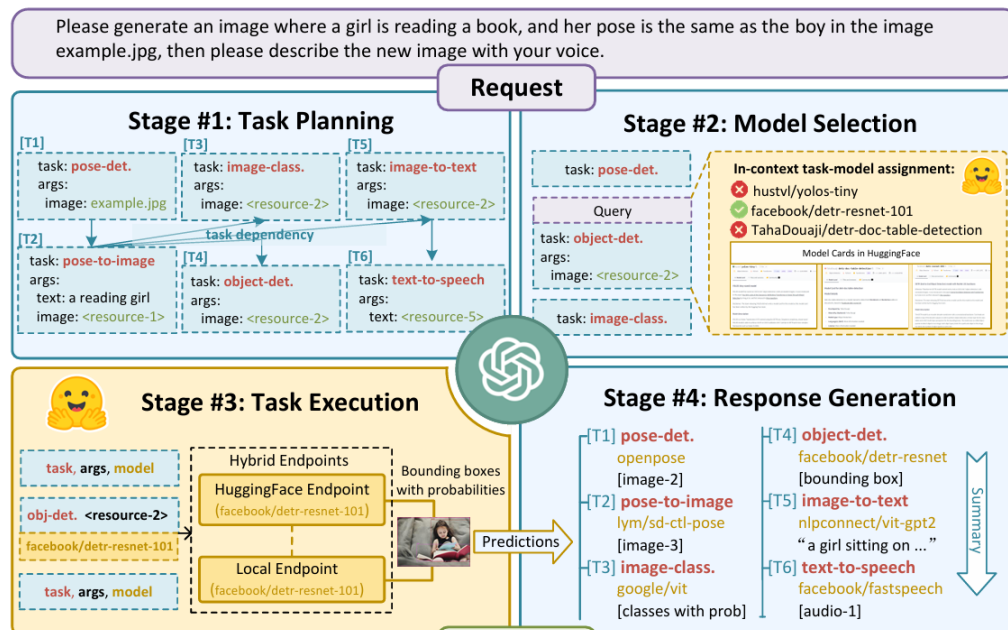
⁸⁰frozen

⁸¹Dataset and Model Repositories

⁸²modalities

⁸³similarity-based transfer

⁸⁴metadata-driven augmentation



شکل ۳-۶: چارچوب HuggingGPT که از مخازن مدل HuggingFace برای خودکارسازی وظایف یادگیری ماشین بهره می‌گیرد [۳۴].

۳-۴-۳ راهبردهای تقویت آمیخته^{۸۵}

سامانه‌های پیشرفته روز، غالباً چند منبع دانش را برای بهره‌گیری از قوت‌های مکمل با هم ترکیب می‌کنند. چارچوب‌های چندکارگزاره ممکن است برنامه‌ریزی تقویت‌شده با بازیابی^{۸۶} که دانش ادبیات و مخازن را برای راهبردهای سطح بالا به کار می‌گیرد را با حافظه رویدادی برآمده از گزارش‌های تجربی که اجرای عملی را صیقل می‌دهد، جفت کنند [۳۸، ۳۶]. به‌همین سیاق، تاریخچه آزمون درون‌متنی می‌تواند با تجربه‌های استانداردسازی‌شده بازیابی‌شده غنی شود تا گرم‌آغاز بهینه‌سازی را به‌ویژه در مواجهه با وظایف نو با ارزیابی‌های اولیه محدود تسریع کند [۳۹].

گزینش معماری تقویت دانش، به‌طرز حساس با روش عملیاتی کارگزار برهم‌کنش دارد: دستوردهی تکرارشونده بیشترین سود را از خلاصه‌های فشرده تاریخی می‌برد؛ عملگرهای تکاملی برای نگهداشت تنوع به بایگانی‌های کیفیت – تنوع اتکا دارند؛ و کنترل‌گرهای جریان‌کار^{۸۷}

^{۸۵}Hybrid Augmentation Strategies

^{۸۶}retrieval-augmented planning

^{۸۷}workflow controllers

برای هماهنگ‌سازی مراحل ناهمگون خط لوله، به فراداده ساخت‌یافته نیازمندند. با گسترش ظرفیت‌های پنجره زمینه و پختگی سازوکارهای بازیابی، مرز میان دانش درون‌متنی و بیرونی هرچه بیشتر محو می‌شود و ادغامی غنی‌تر از پیشن‌های آموخته، شواهد تجربی و خبرگی حوزه را امکان‌پذیر می‌سازد.

۳-۵ تحلیل بر مبنای قالب خروجی مدل

۳-۵-۱ خروجی‌های سبک واژه‌نامه‌ای

نمایش‌های ساخت‌یافته کلید-مقدار^{۸۸} کدگذاری بی‌ابهام فراپارامترها یا انتخاب‌های معماری را با خوانایی ماشینی مستقیم فراهم می‌کنند. سامانه‌هایی که پیکربندی‌های قالب JSON تولید می‌کنند، پارامترهایی مانند نرخ یادگیری، اندازه دسته و ابعاد شبکه را مشخص می‌سازند [۲۶، ۳۷]. گونه‌های پیشرفته، استدلال زنجیره تفکر^{۸۹} را پیش از خروجی ساخت‌یافته می‌گنجانند تا کیفیت پیشنهادها را با استدلال میانی صریح بهبود دهند، در حالی‌که مشخصات نهایی همچنان قابل تجزیه باقی می‌ماند. محدودیت اصلی، مقیدشدن اکتشاف به طرحواره‌های ازپیش‌تعریف‌شده^{۹۰} است که می‌تواند کشف الگوهای طراحی نو را محدود کند. شکل ۳-۷ نمونه‌ای از خروجی قالب JSON را نشان می‌دهد.

۳-۵-۲ تولید کد برنامه

خروجی مبتنی بر کد با تکیه بر بیان‌پذیری کامل زبان‌های برنامه‌نویسی، از قیود فضاهای پیکربندی ازپیش‌تعریف‌شده می‌گریزد. چندین سامانه پیاده‌سازی‌های Python از شبکه‌های عصبی و خطوط لوله یادگیری ماشین را به‌صورت برنامه‌های کامل یا مولفه‌های ماژولار تولید می‌کنند و با آزمون‌های واحد خودکار سازگاری را می‌سنجند [۳۵، ۲۹، ۳۰]. برخی، تولید را بر معیارهای هدف با دستوردهی چندنمونه و نمونه‌هایی برگرفته از جمعیت‌های

⁸⁸structured key-value representations

⁸⁹chain-of-thought

⁹⁰predefined schemas

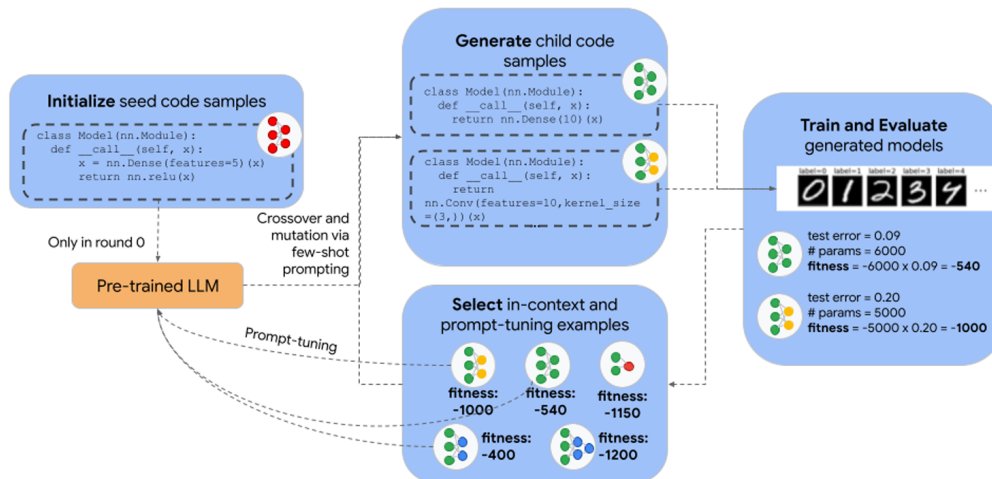
```

Config: {"alpha": 0.1, "batch_size": 8, "depth": 1, "
  ↳ learning_rate_init": 0.0001, "width": 128}
Analysis: The
  ↳ initial configuration with alpha=0.001, batch_size=32,
  ↳ depth=3, learning_rate_init=0.001, and width=64 achieved
  ↳ the lowest loss of 0.058537. The subsequent configuration
  ↳ with alpha=0.01, batch_size=16, depth=2,
  ↳ learning_rate_init=0.01, and width=256 resulted in a
  ↳ higher loss of 0.082927. It seems that decreasing the
  ↳ learning rate and increasing the width did not improve
  ↳ the performance. The latest configuration with alpha=0.1,
  ↳ batch_size=8, depth=1, learning_rate_init=0.0001, and
  ↳ width=128 also resulted in a higher loss of 0.13171.

```

شکل ۳-۷: نمونه‌ای از خروجی قالب JSON برای پیکربندی بهینه‌سازی فرایارامتر در سامانه مبتنی بر مدل زبانی بزرگ [۲۶]

ارزیابی شده شرطی می‌کنند؛ برخی دیگر، عملگرهای تکاملی مانند جهش و ترکیب را مستقیماً بر نمایش‌های کدی اعمال می‌کنند. گونه‌های ترکیبی، تولید کد برای پیاده‌سازی معماری را با فهرست‌های ساخت‌یافته فرایارامتر و متن قالب‌بندی شده گزارش وقایع^{۹۱} آموزشی پیش‌بینی شده درمی‌آمیزند [۳۳، ۳۸]. شکل ۳-۸ نمونه‌ای از خروجی تولید کد را نشان می‌دهد. تولید کد، انعطاف طراحی را به حداکثر می‌رساند و امکان جست‌وجو در معماری‌های



شکل ۳-۸: نمونه‌ای از خروجی تولید کد در سامانه Evoprompting که از مدل زبانی بزرگ برای تولید و بهینه‌سازی کدهای Python شبکه‌های عصبی استفاده می‌کند [۳۰]

نامقید را بدون تعریف صریح اجزای ابتدایی فراهم می‌آورد. با این حال، این رویکرد چالش‌های

^{۹۱}log

اعتبارسنجی به همراه دارد: درستی نحوی^{۹۲} لزوماً آموزش‌پذیری، رعایت قیود منابع^{۹۳} یا معناداری معنایی^{۹۴} را تضمین نمی‌کند. از این رو، سامانه‌ها به محیط‌های اجرا برای ارزیابی نیاز دارند و سازوکارهای مدیریت خطا را برای مواجهه با خروجی‌های نامعتبر پیاده می‌کنند؛ از جمله دستوردهی مجدد^{۹۵} با استفاده از پیام‌های خطا به عنوان بازخورد.

۳-۵-۳ خروجی‌های درخت ساختار

نمایش‌های گراف/درخت بر روابط ترکیبی درون معماری‌ها تأکید می‌کنند و برای وظایفی که به تعیین صریح توپولوژی^{۹۶} نیاز دارند سودمندند؛ بی‌آنکه جزئیات پیاده‌سازی کدی که می‌تواند از استدلال ساختاری منحرف کند تحمیل شود. برخی سامانه‌ها نمایش متنی گراف جهت‌دار بدون دور^{۹۷} با گره‌های شماره‌گذاری شده برای عملیات و اتصالات برمی‌گزینند [۳۶]؛ برخی دیگر از کدگذاری رشته‌ای جداکننده محور^{۹۸} برای ویژگی‌های لایه سازگار با تولید خودبازگشتی^{۹۹} استفاده می‌کنند [۳۲]. خروجی‌ها معمولاً با فرایندهای تجزیه تخصصی به کد اجرایی تبدیل می‌شوند و ابزارهای راستی‌آزمایی گراف محاسباتی^{۱۰۰} علاوه بر صحت نحوی، هم‌ریختی^{۱۰۱} با طرح‌های موجود را نیز می‌سنجند. شکل ۳-۹ نمونه‌ای از خروجی درخت ساختار را نشان می‌دهد. این قالب‌ها استدلال ترکیبی و راهبردهای تغییر سلسله‌مراتبی را تسهیل می‌کنند و به مدل امکان می‌دهند بر توپولوژی معماری مستقل از جزئیات پیاده‌سازی تمرکز کند. به کارگیری آن‌ها به شماهای کدگذاری حوزه‌ای^{۱۰۲} و رویه‌های اعتبارسنجی ویژه نیاز دارد، اما با کاهش پیچیدگی وظیفه تولید از طریق سطح تجرید مناسب، کیفیت تولید را بهبود می‌بخشد.

⁹²syntactic correctness

⁹³resource constraints

⁹⁴semantic meaningfulness

⁹⁵re-prompting

⁹⁶explicit topology specification

⁹⁷Directed Acyclic Graph (DAG)

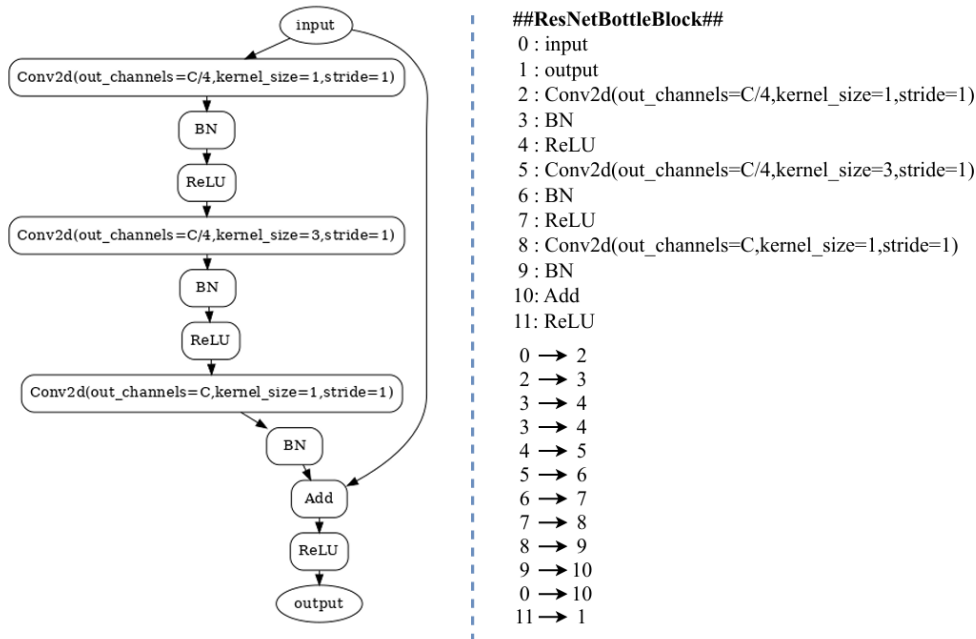
⁹⁸delimited string encodings

⁹⁹autoregressive generation

¹⁰⁰computational graph

¹⁰¹isomorphism

¹⁰²domain-specific encoding schemes

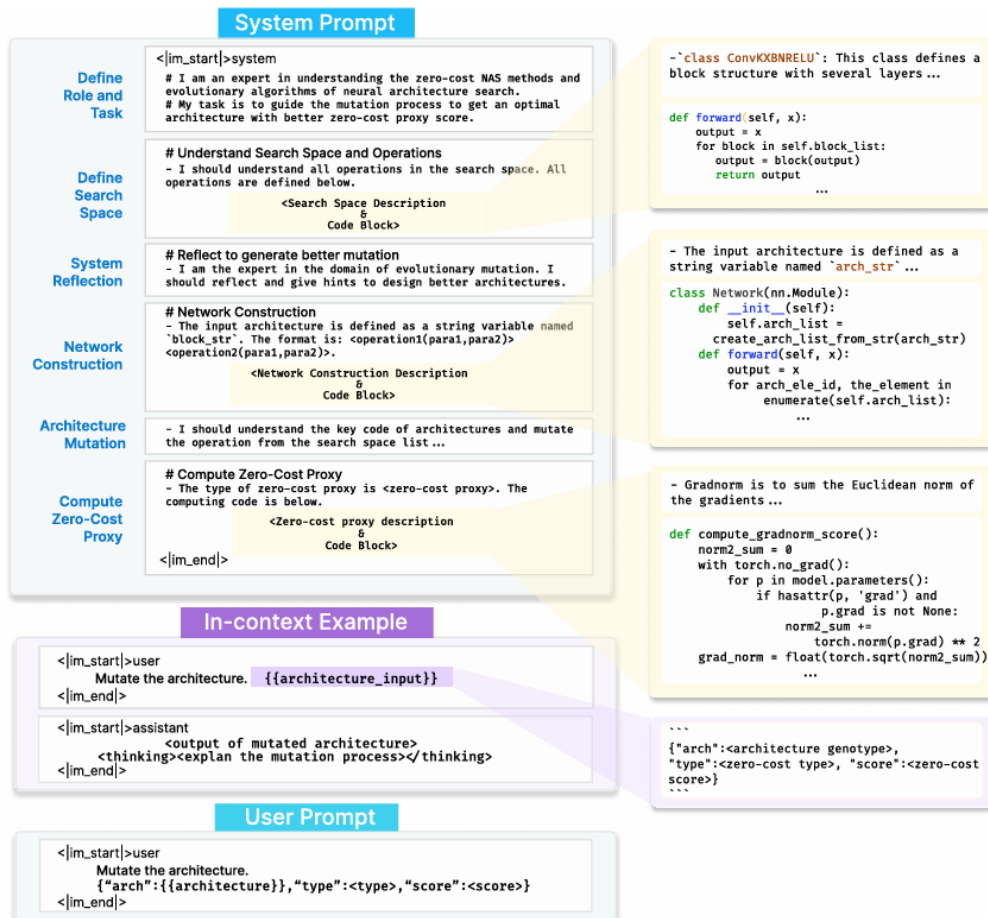


شکل ۳-۹: نمونه‌ای از نمایش گراف محور معماری شبکه عصبی. سمت چپ: تصویرسازی گراف جهت‌دار بدون دور. سمت راست: نمایش متنی گراف جهت‌دار بدون دور برای فهم مدل زبانی بزرگ. [۳۶]

۳-۵-۴ خروجی‌های ترکیبی

سامانه‌های پیشرفته، چندین گونه خروجی^{۱۰۳} را ترکیب می‌کنند تا از قوت‌های مکمل قالب‌های مختلف در مراحل گوناگون خط لوله بهره ببرند. الگوی رایج، مشخصات ساخت‌یافته را با توضیحات زبان طبیعی همراه می‌کند تا هم اجرای ماشینی و هم تفسیر انسانی میسر شود [۳۷، ۲۶]. طرح‌های مفصل‌تر برای مراحل متمایز از قالب‌های متفاوت بهره می‌گیرند: JSON برای نیازمندی‌های قابل اعتبارسنجی صوری، زبان طبیعی برای برنامه‌ریزی و تحلیل منعطف، و کد اجرایی برای پیاده‌سازی‌های نهایی [۳۸، ۳۳]. برخی سامانه‌ها کدگذاری‌های ساختاری فشرده را در توضیحات زبان طبیعی می‌گنجانند تا مشخصات دقیق را با استدلال‌های قابل تفسیر تلفیق کنند [۳۱، ۳۶]. شکل ۳-۱۰ نمونه‌ای از خروجی ترکیبی را نشان می‌دهد. این روش بازتاب این واقعیت است که هیچ قالب یگانه‌ای به‌تنهایی برای همه جنبه‌های یادگیری

¹⁰³ output modalities



شکل ۳-۱۰: نمونه‌ای از خروجی ترکیبی در سامانه RZNAS که از قالب‌های JSON و کد Python برای طراحی معماری شبکه عصبی استفاده می‌کند [۳۱]

ماشین خودکار بهینه نیست: خروجی ساخت‌یافته برای تجزیه و اعتبارسنجی مناسب است؛ کد، پیاده‌سازی منعطف را ممکن می‌سازد؛ گراف‌ها استدلال ترکیبی را تقویت می‌کنند؛ و زبان طبیعی تفسیرپذیری و زمینه غنی را فراهم می‌کند و با موارد دشوار صوری‌سازی روبه‌رو می‌شود. ادغام موفق مستلزم طراحی دقیق گذارهای بین قالب‌ها^{۱۰۴}، رویه‌های اعتبارسنجی میان‌گونه‌ای^{۱۰۵} و راهبردهایی برای مدیریت ناسازگاری‌ها^{۱۰۶} در صورت تعارض نمایش‌ها است.

¹⁰⁴format transitions

¹⁰⁵across modalities

¹⁰⁶inconsistencies

عنوان	عامل	روش سنتی	خروجی	دانش خارجی	فضای جستجو	بدون آموزش	وظیفه
LLM for HPO [۲۶]	تک	—	واژه‌نامه	×	اختیاری	×	HPO
GENIUS [۲۷]	تک	—	واژه‌نامه	×	بله	×	NAS
LLMATIC [۲۹]	تک	EA	کد	×	خیر	×	NAS
LLAMA-NAS [۴۰]	تک	EA	واژه‌نامه	×	بله	×	NAS
Text to ML [۳۵]	چند	—	کد	×	خیر	✓	AutoML
AgentHPO [۳۷]	چند	—	واژه‌نامه	×	بله	×	HPO
AutoML-Agent [۳۸]	چند	—	کد	✓	خیر	✓	AutoML
LLAMBO [۲۸]	چند	BO	متن	×	بله	✓	HPO
Nader [۳۶]	چند	—	درخت	✓	بله	×	NAS
RZ-NAS [۳۱]	تک	EA	کد [†]	×	بله	✓	NAS
EvoPrompting [۳۰]	تک	EA	کد	×	خیر	×	NAS
AutoML-GPT [۳۳]	تک	—	کد	×	بله	×	AutoML
HuggingGPT [۳۴]	چند	—	کد	✓	خیر	×	AutoML
GPT-NAS [۳۲]	تک	EA	درخت	×	خیر	×	NAS
ML Copilot [۳۹]	چند	—	متن	✓	خیر	×	AutoML

جدول ۳-۱: مقایسه فشرده مقالات مبتنی بر LLM. EA=الگوریتم‌های تکاملی، BO=بهینه‌سازی بیزی. نشانه‌ها: [†]=تولید کد + واژه‌نامه.

۳-۶ طبقه بندی کارهای مرتبط

همانطور که در جدول ۳-۱ مشاهده می‌شود، روش‌های بررسی شده در محور معماری عامل تقریباً به طور مساوی بین ساختارهای تک‌عاملی (۸ مورد) و چندعاملی (۷ مورد) تقسیم شده‌اند. برخلاف تصور اولیه، اکثر قریب به اتفاق روش‌ها (۱۱ مورد از ۱۵) در حال حاضر بر دانش درونی (مانند تاریخچه بهینه‌سازی) تکیه می‌کنند و تنها اقلیت کوچکی (۴ مورد) به صراحت از دانش خارجی (مانند مقالات علمی از طریق تولید تقویت‌شده با بازایی) بهره می‌برند.

بیشترین تنوع در قالب خروجی دیده می‌شود؛ جایی که پژوهشگران از قالب‌های گوناگونی متناسب با نیاز مسئله استفاده کرده‌اند. این خروجی‌ها از کدهای برنامه (مناسب برای پیاده‌سازی مستقیم و انعطاف‌پذیری بالا)، واژه‌نامه‌های ساختاریافته (مفید برای پیکربندی‌های صریح)، نمایش‌های درختی (برای تعریف توپولوژی) و حتی متن زبان طبیعی متغیر هستند.

فصل ۴

نتیجه گیری و کارهای آینده

۴-۱ نتیجه گیری

این گزارش سمینار به بررسی و تحلیل رویکردهای نوظهور در یادگیری ماشین خودکار پرداخت که از قابلیت‌های مدل‌های زبانی بزرگ در قالب سیستم‌های عامل-محور بهره می‌برند. مرور ادبیات نشان داد که این حوزه به سرعت در حال فاصله گرفتن از بهینه‌سازهای جعبه‌سیاه سنتی و حرکت به سوی روندهای بر اساس دانش، تفسیرپذیر و خودمختار است. ما روش‌های موجود را از سه منظر کلیدی طبقه‌بندی کردیم: معماری عامل (تک‌عاملی در برابر چندعاملی)، منابع دانش (درونی در برابر بیرونی) و قالب خروجی (ساختاریافته، کد، یا ترکیبی).

یافته‌ها حاکی از آن است که عامل‌های تک‌عاملی، به‌ویژه آن‌هایی که از دستوردهی تکراری یا عملگرهای تکاملی استفاده می‌کنند، برای بهینه‌سازی ابرپارامترها و جستجوی معماری عصبی محدود مؤثر هستند. در مقابل، معماری‌های چندعاملی با تفکیک نقش (مانند پژوهشگر و توسعه‌دهنده)، پتانسیل بیشتری برای مدیریت خطوط لوله پیچیده و بلند-افق یادگیری ماشین خودکار از خود نشان می‌دهند.

ادغام تولید تقویت‌شده با بازیابی یک پیشرفت کلیدی است که به عامل‌ها اجازه می‌دهد

تا از دانش ایستای خود فراتر رفته و از ادبیات پژوهشی، مخازن کد^۱ و نتایج آزمایش‌های گذشته برای اتخاذ تصمیمات آگاهانه‌تر استفاده کنند. در نهایت، قالب خروجی نشان‌دهنده یک توازن میان خوانایی ماشینی^۲ (مانند JSON) و بیانگری (مانند تولید کد کامل) است، که رویکردهای ترکیبی به عنوان راه‌حلی میانه در حال ظهور هستند. در مجموع، یادگیری ماشین خودکار عامل-محور یک حوزه تحقیقاتی بسیار پویا است که نویدبخش خودکارسازی هوشمندانه‌تر و کارآمدتر فرایندهای علم داده است.

۲-۴ مسائل باز و کارهای قابل انجام

حوزه یادگیری ماشین خودکار مبتنی بر عامل‌های زبانی، با وجود پیشرفت‌های سریع، همچنان در مراحل اولیه تکامل خود قرار دارد و مملو از مسائل باز و زمینه‌های پژوهشی است. بر اساس تحلیل‌های ارائه‌شده در این گزارش، می‌توان کارهای آتی را در چند محور اصلی دسته‌بندی کرد:

بهینه‌سازی برای منابع محدود و افزایش کارایی: اکثر روش‌های فعلی بر مدل‌های زبانی بزرگ و پرهزینه (مانند خانواده‌های GPT, Claude, Gemini) متکی هستند. یک مسئله باز کلیدی، تطبیق این رویکردها برای سناریوهایی با منابع محاسباتی محدود است. کارهای آینده می‌تواند شامل تحقیق بر روی ریزتنظیم کردن مدل‌های زبانی با پارامتر کم (مثلاً Gemma-3-12B یا مدل‌های کوچک‌تر) برای وظایف خاص یادگیری ماشین خودکار مانند بهینه‌سازی ابرپارامترها باشد تا به جای اتکای صرف به یادگیری درون-متنی، دانش تخصصی مستقیماً در پارامترهای مدل تزریق شود؛ همچنین استفاده از تکنیک‌های تقطیر دانش^۳ برای انتقال قابلیت‌های استدلال یک مدل زبانی بزرگ به یک مدل کوچک‌تر و کارآمدتر که بتواند به عنوان یک عامل یادگیری ماشین خودکار سبک عمل کند، مسیر مهمی به شمار می‌رود. یکپارچه‌سازی و مدیریت دانش پیشرفته: همانطور که در تحلیل‌ها مشاهده شد، بسیاری از سیستم‌ها هنوز به دانش درونی (تاریخچه بهینه‌سازی) محدود هستند. غنی‌سازی عامل‌ها

¹code repositories

²machine-readability

³knowledge distillation

با دانش خارجی یک مرز پژوهشی مهم است، از جمله طراحی پایگاه‌های دانش پویا و خود-بهبودگر که نه تنها مقالات علمی را شامل شوند، بلکه از مخازن کد (مانند GitHub)، بحث‌های فنی (مانند Stack Overflow) و نتایج بنچمارک‌های عمومی (مانند Papers with Code) نیز تغذیه شوند؛ نیز توسعه راهبردهایی برای مدیریت دانش متناقض، به این صورت که اگر یک مقاله روشی را پیشنهاد دهد اما نتایج تجربی عامل چیز دیگری را نشان دهد، عامل بتواند این تضاد را حل کند.

سیستم‌های چندعاملی با تیم‌های پژوهشی موازی (مسابقه عامل‌ها برای ریزتنظیم): طراحی یک چارچوب که در آن چند «تیم» عامل (با نقش‌های متمایز مانند پژوهشگر، مهندس داده و مربی) هر کدام یک مدل را برای یک تسک مشخص فاین‌تیون کنند و در نهایت بهترین مدل انتخاب یا تجمیع شود، مستلزم وجود هماهنگ‌کننده مرکزی برای تشکیل تیم‌ها، تخصیص نقش‌ها، تعریف قرارداد رابط (مشخصات داده، بودجه، محدودیت‌ها) و زمان‌بندی اجرای آزمایش‌ها است؛ علاوه بر این، باید راهبرد انتخاب یا تجمیع مشخص شود تا بر اساس معیارهای چندهدفه (دقت، زمان، حافظه) بهترین مدل انتخاب گردد و در صورت نزدیک بودن عملکردها، Ensembling سبک (مثلاً logit averaging) و تحلیل جبهه پارتو ارزیابی شود؛ همچنین اشتراک دانش بین تیم‌ها از طریق حافظه مشترک مبتنی بر بازیابی (RAG) برای دسترسی به یافته‌ها، تنظیمات موفق و خطاهای رایج و استانداردسازی ثبت آزمایش‌ها جهت تکرارپذیری اهمیت دارد؛ و نهایتاً تضمین ایمنی و بازتولیدپذیری با تثبیت بذر تصادفی، قفل کردن نسخه وابستگی‌ها، نظارت بر نشستی داده و تعریف آزمون‌های سلامت برای جلوگیری از خطاهای کدنویسی عامل‌ها ضروری است.

سامانه عامل‌محور برای انتخاب و تنظیم بهینه مدل در شرایط . تمرکز بر سه تصمیم کلیدی در انتقال یادگیری است: انتخاب مدل پایه، استراتژی ریزتنظیم و داده‌افزایی؛ و در این میان، تولید یک طرح پیکربندی ساختاریافته (JSON) به عنوان خروجی میانی پیشنهاد می‌شود. معماری چندعاملی شامل عامل تحلیل‌گر دیتاست، عامل استراتژیست برای برنامه‌ریزی و عامل اجراکننده است که به موتور اجرا متصل می‌شود تا پیکربندی تولیدشده را به کد قابل اعتماد تبدیل کند.

۳-۴ معرفی موضوع مورد نظر برای پایان نامه

با توجه به تحلیل‌های صورت گرفته در این سمینار و بررسی مسائل باز موجود، موضوع زیر که بر اساس ایده سوم پیشنهادی تدوین شده است، به عنوان یک حوزه پژوهشی نوآورانه، کاربردی و دارای عمق کافی برای یک پایان‌نامه کارشناسی ارشد انتخاب می‌گردد.

عنوان پیشنهادی

طراحی و پیاده‌سازی یک سیستم یادگیری ماشین خودکار عامل – محور برای انتخاب و تنظیم بهینه مدل

در بسیاری از کاربردهای عملی یادگیری عمیق، دسترسی به داده‌های برچسب‌دار انبوه امکان‌پذیر نیست. در چنین شرایطی، رویکرد غالب، استفاده از انتقال یادگیری^۴ از طریق ریزتنظیم کردن مدل‌های از پیش آموزش دیده است. با این حال، موفقیت این رویکرد به شدت به مجموعه‌ای از تصمیمات پیچیده و به هم وابسته بستگی دارد:

انتخاب مدل پایه^۵: کدام مدل از میان ده‌ها مدل موجود در استخر مدل‌ها (مثلاً ResNet،

EfficientNet، ViT) برای دیتاست و تسک مورد نظر مناسب‌تر است؟

انتخاب استراتژی ریزتنظیم: آیا باید کل مدل را ریزتنظیم کرد، فقط لایه‌های آخر را آموزش داد، یا از روش‌های کارآمد پارامتری مانند LoRA و دیگر تکنیک‌های PEFT استفاده نمود؟

انتخاب روش‌های پیش پردازش و داده‌افزایی: کدام تکنیک‌های داده‌افزایی (مانند CutMix،

Mixup، RandAugment) با توجه به ویژگی‌های دیتاست، بیشترین بهبود را در عملکرد مدل ایجاد می‌کنند؟

فضای جستجوی حاصل از ترکیب این انتخاب‌ها بسیار بزرگ و گسسته است و روش‌های سنتی یادگیری ماشین خودکار برای کاوش مؤثر در آن با چالش مواجه هستند. این پژوهش قصد دارد با بهره‌گیری از یک سیستم چندعاملی مبتنی بر مدل زبانی بزرگ، این فرآیند

^۴Transfer Learning

^۵backbone

تصمیم‌گیری را خودکار و هوشمند سازد. هدف اصلی، ساخت عاملی است که بتواند با تحلیل ویژگی‌های دیتاست ورودی، یک طرح اجرایی^۶ بهینه تولید کند که بهترین ترکیب از مدل، روش ریزتنظیم و تکنیک‌های داده‌افزایی را برای دستیابی به حداکثر دقت با حداقل منابع ممکن، مشخص نماید.

طراحی معماری عامل-محور: یک گردش کار مبتنی بر عامل‌های مدل زبانی بزرگ طراحی می‌شود که وظایف را به صورت منطقی تقسیم کند؛ این معماری می‌تواند شامل یک "عامل تحلیل‌گر" برای استخراج فراداده از دیتاست، یک "عامل استراتژیست" برای تولید طرح پیکربندی بر اساس تحلیل‌ها و دانش پیشین، و یک "عامل اجراکننده" برای اجرای کد مبتنی بر کانفیگ تولید شده باشد.

توسعه مکانیزم تصمیم‌گیری مبتنی بر مدل زبانی بزرگ: مهندسی دستور^۷ و طراحی ساختار ورودی/خروجی به گونه‌ای صورت می‌گیرد که مدل زبانی بزرگ بتواند بر اساس ویژگی‌های دیتاست (مانند اندازه، تعداد کلاس‌ها، نوع داده) و محدودیت‌های منابع، استدلال کرده و تصمیمات آگاهانه بگیرد. خروجی مدل زبانی بزرگ یک فایل پیکربندی ساختاریافته (مثلاً در قالب JSON) خواهد بود.

پیاده‌سازی پل ارتباطی بین استدلال و اجرا: یک موتور اجرایی ساخته می‌شود که فایل پیکربندی تولیدشده توسط مدل زبانی بزرگ را تفسیر کرده و آن را به کد پایتون قابل اجرا تبدیل و اجرا نماید. این رویکرد، استدلال سطح بالای مدل زبانی بزرگ را از اجرای سطح پایین و مستعد خطای کد جدا می‌کند.

ارزیابی جامع سیستم: عملکرد سیستم پیشنهادی بر روی چندین دیتاست بنچمارک با اندازه‌های متفاوت ارزیابی می‌شود و نتایج (دقت و هزینه محاسباتی) با روش‌های پایه مانند انتخاب تصادفی، یک استراتژی ریزتنظیم ثابت و در صورت امکان، سایر ابزارهای یادگیری ماشین خودکار مقایسه خواهد شد.

این پژوهش در چند جنبه دارای نوآوری است: (۱) به جای تمرکز بر یک جزء منفرد مانند بهینه‌سازی ابرپارامترها یا جستجوی معماری عصبی، یک خط لوله کامل و یکپارچه

^۶Configuration Plan

^۷Prompt Engineering

فصل ۴. نتیجه گیری و کارهای آینده ۳-۴. معرفی موضوع مورد نظر برای پایان نامه

برای انتقال یادگیری را هدف قرار می‌دهد. (۲) از مدل زبانی بزرگ نه به عنوان یک بهینه‌ساز جعبه-سیاه، بلکه به عنوان یک موتور استدلال و برنامه‌ریزی برای تولید یک طرح اجرایی شفاف و قابل تفسیر استفاده می‌کند. (۳) استفاده از یک فایل پیکربندی به عنوان خروجی میانی، یک راهکار نوین برای ترکیب قدرت استدلال مدل زبانی بزرگ با قابلیت اطمینان و استحکام سیستم‌های نرم‌افزاری کدمحور است. موفقیت این پروژه می‌تواند گام مهمی در جهت "مردمی‌سازی" استفاده بهینه از مدل‌های پایه باشد و به محققان و مهندسان کمک کند تا با سرعت و کارایی بیشتری به نتایج مطلوب دست یابند.

کتاب نامه

- [1] I. Salehin, M. S. Islam, P. Saha, S. Noman, A. Tuni, M. M. Hasan, and M. A. Baten, “Automl: A systematic review on automated machine learning with neural architecture search,” *Journal of Information and Intelligence*, vol.2, no.1, pp.52–81, 2024.
- [2] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, “Auto-weka: combined selection and hyperparameter optimization of classification algorithms,” KDD '13, (New York, NY, USA), p.847–855, Association for Computing Machinery, 2013.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol.30, 2017.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp.4171–4186, 2019.
- [5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol.33, pp.1877–1901, 2020.
- [6] L. Wang, C. Lyu, T. Ji, Z. Zhang, D. Yu, S. Shi, and Z. Tu, “Document-level machine translation with large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (H. Bouamor, J. Pino, and K. Bali, eds.), (Singapore), pp.16646–16661, Association for Computational Linguistics, Dec. 2023.
- [7] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “Pal: Program-aided language models,” in *International Conference on Machine Learning*, pp.10764–10799, PMLR, 2023.

- [8] L. Pan, A. Albalak, X. Wang, and W. Wang, “Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning,” in *Findings of the Association for Computational Linguistics: EMNLP 2023* (H. Bouamor, J. Pino, and K. Bali, eds.), (Singapore), pp.3806–3824, Association for Computational Linguistics, Dec. 2023.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [10] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol.18, no.6, p.186345, 2024.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS ’20*, (Red Hook, NY, USA), Curran Associates Inc., 2020.
- [12] Y. Xia, J. Zhou, Z. Shi, J. Chen, and H. Huang, “Improving retrieval augmented language model with self-reasoning,” in *Proceedings of the AAAI conference on artificial intelligence*, vol.39, pp.25534–25542, 2025.
- [13] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, vol.2, no.1, 2023.
- [14] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, “Agentic retrieval-augmented generation: A survey on agentic rag,” *arXiv preprint arXiv:2501.09136*, 2025.
- [15] J. R. Rice, “The algorithm selection problem**this work was partially supported by the national science foundation through grant gp-32940x. this chapter was presented as the george e. forsythe memorial lecture at the computer science conference, february 19, 1975, washington, d. c.,” vol.15 of *Advances in Computers*, pp.65–118, Elsevier, 1976.
- [16] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol.13, no.10, pp.281–305, 2012.

- [17] F. Hutter, H. H. Hoos, and K. Leyton-Brown, “Sequential model-based optimization for general algorithm configuration,” in *Learning and Intelligent Optimization* (C. A. C. Coello, ed.), (Berlin, Heidelberg), pp.507–523, Springer Berlin Heidelberg, 2011.
- [18] M. Feurer, A. Klein, K. Eggenberger, J. Springenberg, M. Blum, and F. Hutter, “Efficient and robust automated machine learning,” in *Advances in Neural Information Processing Systems* (C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, eds.), vol.28, Curran Associates, Inc., 2015.
- [19] R. S. Olson and J. H. Moore, “Tpot: A tree-based pipeline optimization tool for automating machine learning,” in *Proceedings of the Workshop on Automatic Machine Learning* (F. Hutter, L. Kotthoff, and J. Vanschoren, eds.), vol.64 of *Proceedings of Machine Learning Research*, (New York, New York, USA), pp.66–74, PMLR, 24 Jun 2016.
- [20] K. Jamieson and A. Talwalkar, “Non-stochastic best arm identification and hyperparameter optimization,” in *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics* (A. Gretton and C. C. Robert, eds.), vol.51 of *Proceedings of Machine Learning Research*, (Cadiz, Spain), pp.240–248, PMLR, 09–11 May 2016.
- [21] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” *Journal of Machine Learning Research*, vol.18, no.185, pp.1–52, 2018.
- [22] B. Bischl, G. Casalicchio, M. Feurer, P. Gijsbers, F. Hutter, M. Lang, R. G. Mantovani, J. N. van Rijn, and J. Vanschoren, “Openml benchmarking suites,” 2021.
- [23] B. Zoph and Q. Le, “Neural architecture search with reinforcement learning,” in *International Conference on Learning Representations*, 2017.
- [24] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, “Efficient neural architecture search via parameters sharing,” in *Proceedings of the 35th International Conference on Machine Learning* (J. Dy and A. Krause, eds.), vol.80 of *Proceedings of Machine Learning Research*, pp.4095–4104, PMLR, 10–15 Jul 2018.
- [25] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter, “NAS-bench-101: Towards reproducible neural architecture search,” in *Proceedings of the 36th International Conference on Machine Learning* (K. Chaudhuri and R. Salakhutdinov, eds.), vol.97 of *Proceedings of Machine Learning Research*, pp.7105–7114, PMLR, 09–15 Jun 2019.

- [26] M. Zhang, N. Desai, J. Bae, J. Lorraine, and J. Ba, “Using large language models for hyperparameter optimization,” in *NeurIPS 2023 Foundation Models for Decision Making Workshop*, 2023.
- [27] M. Zheng, X. Su, S. You, F. Wang, C. Qian, C. Xu, and S. Albanie, “Can gpt-4 perform neural architecture search?,” 2023.
- [28] T. Liu, N. Astorga, N. Seedat, and M. van der Schaar, “Large language models to enhance bayesian optimization,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [29] M. U. Nasir, S. Earle, J. Togelius, S. James, and C. Cleghorn, “Llmatic: Neural architecture search via large language models and quality diversity optimization,” in *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO ’24*, (New York, NY, USA), p.1110–1118, Association for Computing Machinery, 2024.
- [30] A. Chen, D. Dohan, and D. So, “Evoprompting: Language models for code-level neural architecture search,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [31] Z. Ji, G. Zhu, C. Yuan, and Y. Huang, “RZ-NAS: Enhancing LLM-guided neural architecture search via reflective zero-cost strategy,” in *Forty-second International Conference on Machine Learning*, 2025.
- [32] C. Yu, X. Liu, Y. Wang, Y. Liu, W. Feng, X. Deng, C. Tang, and J. Lv, “Gpt-nas: Neural architecture search meets generative pre-trained transformer model,” *Big Data Mining and Analytics*, vol.8, no.1, pp.45–64, 2025.
- [33] S. Zhang, C. Gong, L. Wu, X. Liu, and M. Zhou, “Automl-gpt: Automatic machine learning with gpt,” 2023.
- [34] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface,” in *Advances in Neural Information Processing Systems*, 2023.
- [35] J. Xu, J. Li, Z. Liu, N. Suryanarayanan, G. Zhou, J. GUO, H. Iba, and K. Tei, “Large language models synergize with automated machine learning,” *Transactions on Machine Learning Research*, 2024.

- [36] Z. Yang, W. Zeng, S. Jin, C. Qian, P. Luo, and W. Liu, “Nader: Neural architecture design via multi-agent collaboration,” in *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp.4452–4461, June 2025.
- [37] S. Liu, C. Gao, and Y. Li, “AgentHPO: Large language model agent for hyper-parameter optimization,” in *The Second Conference on Parsimony and Learning (Proceedings Track)*, 2025.
- [38] P. Trirat, W. Jeong, and S. J. Hwang, “AutoML-agent: A multi-agent LLM framework for full-pipeline autoML,” in *Forty-second International Conference on Machine Learning*, 2025.
- [39] L. Zhang, Y. Zhang, K. Ren, D. Li, and Y. Yang, “MLCopilot: Unleashing the power of large language models in solving machine learning tasks,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (Y. Graham and M. Purver, eds.), (St. Julian’s, Malta), pp.2931–2959, Association for Computational Linguistics, Mar. 2024.
- [40] A. Sarah, S. Nittur Sridhar, M. Szankin, and S. Sundaresan, “Llama-nas: Efficient neural architecture search for large language models,” in *European Conference on Computer Vision*, pp.67–74, Springer, 2024.

واژه‌نامه فارسی به انگلیسی

trials	آزمون‌ها
automated unit tests	آزمون‌های واحد خودکار
context-aware	آگاه از بافتار
Hyperparameters	ابریارامترها
tool	ابزار
skip connections	اتصالات پرشی
literature	ادبیات پژوهشی
robustness	استحکام
offline knowledge elicitation	استخراج دانش برون خط
Literature-Driven Knowledge Extraction	استخراج دانش مبتنی بر ادبیات پژوهشی
retrieval-augmented reasoning	استدلال تقویت شده با بازیابی
documents	اسناد
parameter sharing	اشتراک پارامتر
post-validation	اعتبارسنجی پسین
genetic algorithm	الگوریتم ژنتیک
evolutionary algorithms	الگوریتم‌های تکاملی
search algorithms	الگوریتم‌های جستجو
few-shot demonstrations	الگوهای اندک نمونه
relevance scoring	امتیازدهی ارتباط
model selection	انتخاب مدل
similarity-based transfer	انتقال مبتنی بر شباهت
Transfer Learning	انتقال یادگیری
linguistic feedback	بازخورد زبانی
actionable feedback	بازخورد قابل اقدام
reconstructor	بازساز
similarity-based retrieval	بازیابی مبتنی بر شباهت

semantic similarity retrieval	بازیابی مبتنی بر شباهت معنایی
context	بافتار
planning	برنامه‌ریزی
retrieval-augmented planning	برنامه‌ریزی تقویت‌شده با بازیابی
planning/solution generation	برنامه‌ریزی/تولید راه‌حل
genetic programming	برنامه‌نویسی ژنتیکی
long-horizon	بلندافق
optimization	بهینه‌سازی
Hyperparameter Optimization (HPO)	بهینه‌سازی ابرپارامترها
Bayesian optimization	بهینه‌سازی بیزی
up-to-dateness	به‌روز بودن
zero-shot	بی‌نمونه
Knowledge Source Analysis	تحلیل منابع دانش
machine translation	ترجمه ماشینی
crossover	ترکیب
adaptive	تطبیق‌پذیر
interact	تعامل
generalization	تعمیم‌پذیری
explicit topology specification	تعیین صریح توپولوژی
Reasoning	تفکر
task decomposition	تفکیک وظایف
knowledge distillation	تقطیر دانش
metadata-driven augmentation	تقویت مبتنی بر فراداده
hyperparameter tuning	تنظیم ابرپارامترها
prompt-tuning	تنظیم دستور
reflective capabilities	توانایی‌های بازتابی
Retrieval-Augmented Generation (RAG)	تولید تقویت‌شده با بازیابی
autoregressive generation	تولید خودبازگشتی
one-shot solution generation	تولید راه‌حل تک‌نمونه‌ای
code generation	تولید کد
hallucination	توهم
Single-Agent	تک‌عاملی
Development Team	تیم توسعه
Research Team	تیم پژوهش

surrogate	جانشین
embed	جانمایی کردن
embeddings	جانمایی‌ها
Random Search	جستجوی تصادفی
Grid Search	جستجوی شبکه‌ای
Neural Architecture Search (NAS)	جستجوی معماری عصبی
black-box	جعبه‌سیاه
random forests	جنگل‌های تصادفی
mutation	جهش
episodic memory	حافظه ترتیبی
sensors	حسگرها
feedback loops	حلقه‌های بازخورد
revision loops	حلقه‌های بازنگری
crawl	خزش
text summarization	خلاصه‌سازی متن
machine-readability	خوانایی ماشینی
self-attention	خود توجه
self-reflection	خودبازتابی
autonomous	خودمختار
External Knowledge via Retrieval	دانش بیرونی: بازیابی از ادبیات و مخازن
System-Internal Knowledge: Trials and Reflection	دانش درونی: تاریخچه آزمون و بازتاب
syntactic correctness	درستی نحوی
interleaved	درهم‌تنیده
understanding context	درک بافتار
Natural Language Understanding (NLU)	درک زبان طبیعی
classification	دسته‌بندی
prompt	دستور
iterative prompting	دستوردهی تکراری
contextual prompting	دستوردهی زمینه‌مند
re-prompting	دستوردهی مجدد
application programming interfaces (APIs)	رابط‌های برنامه‌نویسی کاربردی
bandit-based strategies	راهبردهای باندیتی
Hybrid Augmentation Strategies	راهبردهای تقویت آمیخته
knowledge augmentation strategies	راهبردهای تقویت دانش

reasoning traces	ردپاهای استدلالی
competitive	رقابتی
functional decomposition	رهگذر تفکیک کارکردی
evolutionary methods	روش‌های تکاملی
fine-tuning	ریزتنظیم
chain-of-thought	زنجیره تفکر
pipeline	زنجیره وظایف
information retrieval mechanism	سازوکار بازیابی اطلاعات
serialization	سریال‌سازی
Agent-based Systems	سیستم‌های عامل – محور
Deep Neural Networks	شبکه‌های عصبی عمیق
warmstart	شروع تازه
domain-specific encoding schemes	شماهای کدگذاری حوزه‌ای
normalization schemes	شماهای نرمال‌سازی
intuition	شهود
Configuration Plan	طرح اجرایی
predefined schemas	طرحواره‌های ازپیش تعریف شده
Agent Manager	عامل مدیر
agent	عامل
Action	عمل
Evolutionary Operators	عملگرهای تکاملی
meta-learning	فرا – یادگیری
structured metadata	فرا داده ساختاریافته
design space	فضای طراحی
parallelizable subtasks	قابل موازی‌سازی
chunks	قطعات
resource constraints	قيود منابع
ensemble	مجمع
corpus	مجموعه متون
environment	محیط
Dataset and Model Repositories	مخازن مجموعه داده و مدل
code repositories	مخازن کد
backbone	مدل پایه
surrogate modeling	مدل‌سازی جانشین

Large Language Models (LLMs)	مدل‌های زبانی بزرگ
democratization	مردمی‌سازی
semantic meaningfulness	معناداری معنایی
scale	مقیاس
critic	منتقد
frozen	منجمد
coherent	منسجم
Prompt Engineering	مهندسی دستور
feature engineering	مهندسی ویژگی
modular components	مولفه‌های ماژولار
across modalities	میان‌گونه‌ای
inconsistencies	ناسازگاری‌ها
vector representations	نمایش‌های برداری
structured key-value representations	نمایش‌های ساخت‌یافته کلید-مقدار
graph representations	نمایش‌های گراف
candidate sampling	نمونه‌برداری نامزد
computational cost	هزینه محاسباتی
Hierarchical Coordination	هماهنگی سلسله‌مراتبی
coordinator	هماهنگ‌کننده
isomorphism	هم‌ریختی
collaborative	همکارانه
inter-agent collaboration	همکاری بین‌عاملی
Role-Based Collaboration	همکاری مبتنی بر نقش
orchestrator	هم‌آوا ساز
long-range dependencies	وابستگی‌های دوربرد
modalities	وجه‌های داده
static responses	پاسخ‌های ایستا
vector database	پایگاه داده برداری
knowledge base	پایگاه دانش
vector databases	پایگاه‌های داده برداری
knowledge bases	پایگاه‌های دانش
communication protocols	پروتکل‌های ارتباطی
context window	پنجره بافتار
pre-training	پیش‌آموزش

data preprocessing	پیش‌پردازش داده
configuration	پیکربندی
paradigm	چارچوب نظری
Multi-Agent	چندعاملی
multi-fidelity	چندوفایی
performance	کارایی
computational efficiency	کارایی محاسباتی
data cards	کارت‌های داده
model cards	کارت‌های مدل
specialized reader agents	کارگزاران خوانش تخصصی
canonicalization	کانونی سازی
encoder	کدگذار
delimited string encodings	کدگذاری رشته‌ای جداکننده‌محور
decoder	کدگشا
few-shot	کم‌نمونه
workflow controllers	کنترل‌گرهای جریان‌کار
actuators	کنشگرها
quality-diversity	کیفیت-تنوع
format transitions	گذارهای بین قالب‌ها
Directed Acyclic Graph (DAG)	گراف جهت‌دار بدون دور
computational graph	گراف محاسباتی
log	گزارش وقایع
feature selectors	گزیننده‌های ویژگی
discretized	گسسته‌سازی
chat-style dialogues	گفت‌وگوهای چت محور
output modalities	گونه خروجی
In-Context Learning (ICL)	یادگیری بافتاری
Reinforcement Learning	یادگیری تقویتی
In-Context Learning from Optimization History	یادگیری درون‌متنی از تاریخچه بهینه‌سازی
Automated Machine Learning (AutoML)	یادگیری ماشین خودکار

واژه‌نامه انگلیسی به فارسی

trials	آزمون‌ها
automated unit tests	آزمون‌های واحد خودکار
context-aware	آگاه از بافتار
Hyperparameters	ابریارامترها
tool	ابزار
skip connections	اتصالات پرشی
literature	ادبیات پژوهشی
robustness	استحکام
offline knowledge elicitation	استخراج دانش برون خط
Literature-Driven Knowledge Extraction	استخراج دانش مبتنی بر ادبیات پژوهشی
retrieval-augmented reasoning	استدلال تقویت شده با بازیابی
documents	اسناد
parameter sharing	اشتراک پارامتر
post-validation	اعتبارسنجی پسین
genetic algorithm	الگوریتم ژنتیک
evolutionary algorithms	الگوریتم‌های تکاملی
search algorithms	الگوریتم‌های جستجو
few-shot demonstrations	الگوهای اندک نمونه
relevance scoring	امتیازدهی ارتباط
model selection	انتخاب مدل
similarity-based transfer	انتقال مبتنی بر شباهت
Transfer Learning	انتقال یادگیری
linguistic feedback	بازخورد زبانی
actionable feedback	بازخورد قابل اقدام
reconstructor	بازساز
similarity-based retrieval	بازیابی مبتنی بر شباهت

semantic similarity retrieval	بازیابی مبتنی بر شباهت معنایی
context	بافتار
planning	برنامه‌ریزی
retrieval-augmented planning	برنامه‌ریزی تقویت‌شده با بازیابی
planning/solution generation	برنامه‌ریزی/تولید راه‌حل
genetic programming	برنامه‌نویسی ژنتیکی
long-horizon	بلندافق
optimization	بهینه‌سازی
Hyperparameter Optimization (HPO)	بهینه‌سازی ابرپارامترها
Bayesian optimization	بهینه‌سازی بیزی
up-to-dateness	به‌روز بودن
zero-shot	بی‌نمونه
Knowledge Source Analysis	تحلیل منابع دانش
machine translation	ترجمه ماشینی
crossover	ترکیب
adaptive	تطبیق‌پذیر
interact	تعامل
generalization	تعمیم‌پذیری
explicit topology specification	تعیین صریح توپولوژی
Reasoning	تفکر
task decomposition	تفکیک وظایف
knowledge distillation	تقطیر دانش
metadata-driven augmentation	تقویت مبتنی بر فراداده
hyperparameter tuning	تنظیم ابرپارامترها
prompt-tuning	تنظیم دستور
reflective capabilities	توانایی‌های بازتابی
Retrieval-Augmented Generation (RAG)	تولید تقویت‌شده با بازیابی
autoregressive generation	تولید خودبازگشتی
one-shot solution generation	تولید راه‌حل تک‌نمونه‌ای
code generation	تولید کد
hallucination	توهم
Single-Agent	تک‌عاملی
Development Team	تیم توسعه
Research Team	تیم پژوهش

surrogate	جانشین
embed	جانمایی کردن
embeddings	جانمایی‌ها
Random Search	جستجوی تصادفی
Grid Search	جستجوی شبکه‌ای
Neural Architecture Search (NAS)	جستجوی معماری عصبی
black-box	جعبه سیاه
random forests	جنگل‌های تصادفی
mutation	جهش
episodic memory	حافظه ترتیبی
sensors	حسگرها
feedback loops	حلقه‌های بازخورد
revision loops	حلقه‌های بازنگری
crawl	خزش
text summarization	خلاصه‌سازی متن
machine-readability	خوانایی ماشینی
self-attention	خود توجه
self-reflection	خودبازتابی
autonomous	خودمختار
External Knowledge via Retrieval	دانش بیرونی: بازیابی از ادبیات و مخازن
System-Internal Knowledge: Trials and Reflection	دانش درونی: تاریخچه آزمون و بازتاب
syntactic correctness	درستی نحوی
interleaved	درهم‌تنیده
understanding context	درک بافتار
Natural Language Understanding (NLU)	درک زبان طبیعی
classification	دسته‌بندی
prompt	دستور
iterative prompting	دستوردهی تکراری
contextual prompting	دستوردهی زمینه‌مند
re-prompting	دستوردهی مجدد
application programming interfaces (APIs)	رابط‌های برنامه‌نویسی کاربردی
bandit-based strategies	راهبردهای باندیتی
Hybrid Augmentation Strategies	راهبردهای تقویت آمیخته
knowledge augmentation strategies	راهبردهای تقویت دانش

reasoning traces	ردپاهای استدلالی
competitive	رقابتی
functional decomposition	رهگذر تفکیک کارکردی
evolutionary methods	روش‌های تکاملی
fine-tuning	ریزتنظیم
chain-of-thought	زنجیره تفکر
pipeline	زنجیره وظایف
information retrieval mechanism	سازوکار بازیابی اطلاعات
serialization	سریال‌سازی
Agent-based Systems	سیستم‌های عامل – محور
Deep Neural Networks	شبکه‌های عصبی عمیق
warmstart	شروع تازه
domain-specific encoding schemes	شماهای کدگذاری حوزه‌ای
normalization schemes	شماهای نرمال‌سازی
intuition	شهود
Configuration Plan	طرح اجرایی
predefined schemas	طرحواره‌های ازپیش تعریف شده
Agent Manager	عامل مدیر
agent	عامل
Action	عمل
Evolutionary Operators	عملگرهای تکاملی
meta-learning	فرا – یادگیری
structured metadata	فرا داده ساختاریافته
design space	فضای طراحی
parallelizable subtasks	قابل موازی‌سازی
chunks	قطعات
resource constraints	قيود منابع
ensemble	مجمع
corpus	مجموعه متون
environment	محیط
Dataset and Model Repositories	مخازن مجموعه داده و مدل
code repositories	مخازن کد
backbone	مدل پایه
surrogate modeling	مدل‌سازی جانشین

Large Language Models (LLMs)	مدل‌های زبانی بزرگ
democratization	مردمی‌سازی
semantic meaningfulness	معناداری معنایی
scale	مقیاس
critic	منتقد
frozen	منجمد
coherent	منسجم
Prompt Engineering	مهندسی دستور
feature engineering	مهندسی ویژگی
modular components	مولفه‌های ماژولار
across modalities	میان‌گونه‌ای
inconsistencies	ناسازگاری‌ها
vector representations	نمایش‌های برداری
structured key-value representations	نمایش‌های ساخت‌یافته کلید-مقدار
graph representations	نمایش‌های گراف
candidate sampling	نمونه‌برداری نامزد
computational cost	هزینه محاسباتی
Hierarchical Coordination	هماهنگی سلسله‌مراتبی
coordinator	هماهنگ‌کننده
isomorphism	هم‌ریختی
collaborative	همکارانه
inter-agent collaboration	همکاری بین‌عاملی
Role-Based Collaboration	همکاری مبتنی بر نقش
orchestrator	هم‌آوا ساز
long-range dependencies	وابستگی‌های دوربرد
modalities	وجه‌های داده
static responses	پاسخ‌های ایستا
vector database	پایگاه داده برداری
knowledge base	پایگاه دانش
vector databases	پایگاه‌های داده برداری
knowledge bases	پایگاه‌های دانش
communication protocols	پروتکل‌های ارتباطی
context window	پنجره بافتار
pre-training	پیش‌آموزش

data preprocessing	پیش‌پردازش داده
configuration	پیکربندی
paradigm	چارچوب نظری
Multi-Agent	چندعاملی
multi-fidelity	چندوفایی
performance	کارایی
computational efficiency	کارایی محاسباتی
data cards	کارت‌های داده
model cards	کارت‌های مدل
specialized reader agents	کارگزاران خوانش تخصصی
canonicalization	کانونی سازی
encoder	کدگذار
delimited string encodings	کدگذاری رشته‌ای جداکننده‌محور
decoder	کدگشا
few-shot	کم‌نمونه
workflow controllers	کنترل‌گرهای جریان‌کار
actuators	کنشگرها
quality-diversity	کیفیت-تنوع
format transitions	گذارهای بین قالب‌ها
Directed Acyclic Graph (DAG)	گراف جهت‌دار بدون دور
computational graph	گراف محاسباتی
log	گزارش وقایع
feature selectors	گزیننده‌های ویژگی
discretized	گسسته‌سازی
chat-style dialogues	گفت‌وگوهای چت محور
output modalities	گونه خروجی
In-Context Learning (ICL)	یادگیری بافتاری
Reinforcement Learning	یادگیری تقویتی
In-Context Learning from Optimization History	یادگیری درون‌متنی از تاریخچه بهینه‌سازی
Automated Machine Learning (AutoML)	یادگیری ماشین خودکار

Abstract:

Traditional AutoML approaches using Bayesian optimization and evolutionary algorithms lack interpretability and struggle to leverage domain knowledge. Large Language Models (LLMs) with capabilities in natural language understanding, reasoning, and code generation offer a transformative paradigm shift. This seminar explores how LLM-based agents can revolutionize AutoML systems by acting as intelligent, context-aware, and adaptive agents rather than blind optimizers. This work systematically reviews recent LLM-based AutoML approaches from three perspectives: (1) Agent Architecture: single-agent systems (direct optimization, evolutionary operators, workflow controllers) versus multi-agent systems (role-based collaboration, hierarchical coordination) (2) Knowledge Sources: internal knowledge (optimization history, in-context learning) versus external retrieval (literature extraction, model repositories) and (3) Output Formats: structured configurations, code generation, tree representations, and hybrid approaches. Analysis reveals an even split between single-agent and multi-agent architectures, with most systems relying on internal knowledge and few leveraging external knowledge through retrieval-augmented generation. Output formats show high diversity, with hybrid approaches balancing machine-readability and expressiveness. The seminar identifies critical open problems and future directions, including resource-constrained optimization, advanced knowledge integration, and multi-agent frameworks. A thesis proposal is presented for designing an agent-based AutoML system for model selection and fine-tuning in transfer learning with limited data.

Keywords: Automated Machine Learning, Large Language Models, Agent-Based Systems, Hyperparameter Optimization, Neural Architecture Search, Retrieval-Augmented Generation, In-Context Learning, Transfer Learning



**Iran University of Science and Technology
Computer Engineering Department**

Review of agent-based large language models in neural architecture search and hyperparameter optimization

**A Seminar Report Submitted in Partial Fulfillment of the Requirements for
the Degree of Master of Science in Computer Engineering-Artificial
Intelligence**

By:

Mohammad Sadegh Poulaei Moziraji

Supervisors:

Dr. MohammadReza Mohammadi and Dr. Sauleh Etemadi

November 2025